

---

# A Brief Summary of Optimization in Deep Learning in New Era

---

**Yifeng Liu**

University of California, Los Angeles  
liuyifeng@g.ucla.edu

## Abstract

Newton’s method provides one of the earliest insight in optimization theories. Based on gradient descent, a lot of optimization theories including momentum, adaptive learning, sign of gradient, second-order optimization, variance reduction and scheduler-free optimization have been proposed. However, there is a lack of comprehensive and clear summary of these approaches with unified notation system. This paper attempts to give a systematic, explicit and concise formulation of over 180 optimization methods in deep learning with citation, which is potential for promoting innovation in optimization theory in deep learning, while facilitating relevant researchers to search for references. And the related materials can be found in <https://github.com/lauyikfung/A-Summary-Sheet-of-Optimization-in-Deep-Learning>.

## 1 Introduction

Optimization lies at the heart of deep learning, facilitating the training process of deep neural networks nowadays, including large language models (Team et al., 2025; Liu et al., 2024a; Grattafiori et al., 2024)(Figure 1), and computer vision models (Ramesh et al., 2022; Liu et al., 2023). From the foundational principles of Newton’s method, numerous optimization theories has emerged, significantly enhancing the efficiency and effectiveness of training deep neural networks. These advancements encompass a wide array of techniques, including momentum-based methods, adaptive learning rate strategies, approaches leveraging the sign of gradients, second-order optimization techniques, variance reduction schemes, and even scheduler-free optimization paradigms. Despite the proliferation of these innovative methods, a comprehensive and clearly structured summary, particularly one employing a unified notation system, has been notably absent. This gap often poses a challenge for researchers seeking to navigate the vast and rapidly evolving field of deep learning optimization, hindering both the promotion of new innovations and the efficient discovery of relevant references.

This paper aims to bridge this gap by presenting a systematic, explicit, and unified formulation of more than 180 optimization methods for deep learning. Each method is documented with appropriate citations, aiming to provide a singular, accessible resource for the research community. By offering a unified framework and clear descriptions, this work seeks to foster further innovation in optimization theory within deep learning and to significantly streamline the process for researchers to identify and utilize relevant methodologies.

It is worth noting that to provide a clear view of the existing optimization algorithms in deep learning, some methods designed for fields including federated learning, simple convex problems, low-rank adaptation (LoRA), reinforcement learning, hardware-efficient improvement, multi-modal models specifics, and methods with strong assumptions or ill-defined variants are not included in this paper.

# A Brief History of LLMs

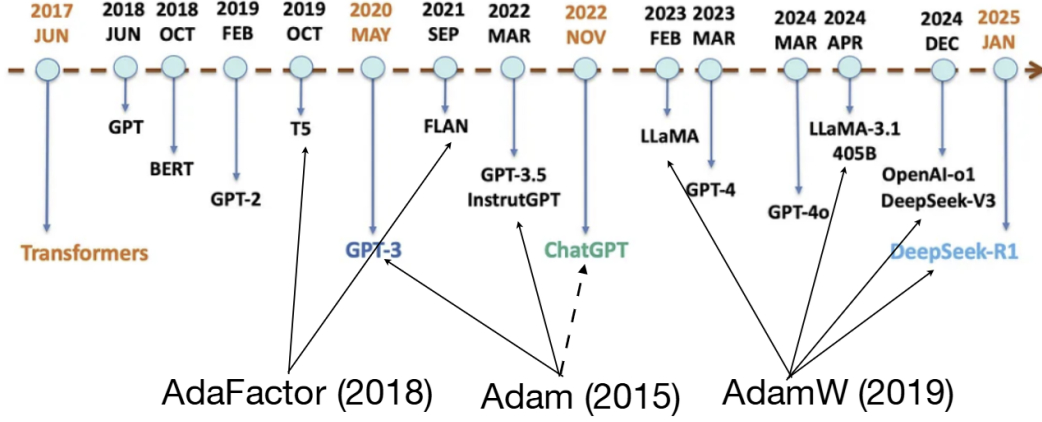


Figure 1: A brief history of large language models. Some optimizers are used in training well-known large language models. The base figure is from <https://medium.com/@lmpo/a-brief-history-of-lmms-from-transformers-2017-to-deepseek-r1-2025-dae75dd3f59a>

## 2 From Gradient Descent to Adaptive Learning

### 2.1 Notations

In the paper,  $f(\theta_t)$  denotes the deep learning network at  $t$ -th iteration with parameter  $\theta_t$ , which is usually regarded as a vector except for second-order methods. The objective is  $J(\theta_t)$  and the gradient is  $\nabla_{\theta} J(\theta_t) = \nabla f(\theta_t, \xi_t) = g_t$ , where the input is  $\xi_t$  by default. Moreover,  $\|\mathbf{x}\| = \|\mathbf{x}\|_2$  denotes the 2-norm, while  $|\mathbf{x}|$  denotes taking the absolute value element-wisely.  $\eta$  is the learning rate or step size, and it may change over iterations unless specific explanation. We just omit the initialization of parameters and state variables for simplicity.

### 2.2 Newton's Method

For Newton's method,  $f(\theta + \epsilon) = f(\theta) + \epsilon^T \nabla f(\theta) + \frac{1}{2} \epsilon^T \mathbf{H} \epsilon + \mathcal{O}(\|\epsilon\|^3)$ , where  $\mathbf{H} = \nabla^2 f(\theta)$  is the Hessian matrix (Zhang et al., 2021).

For first-order method, the second-order and higher-order terms ( $\frac{1}{2} \epsilon^T \mathbf{H} \epsilon + \mathcal{O}(\|\epsilon\|^3)$ ) are ignored since they have relatively lower influence on the convergence of training and much harder to compute, and the update rule comes to:

$$\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t),$$

where  $\eta = -\epsilon$  is the step size.

While in second-order methods, only the higher-order terms ( $\mathcal{O}(\|\epsilon\|^3)$ ) are ignored. Since at the minimum of  $f$ ,  $\nabla_{\theta} f(\theta) = 0$ , then  $\epsilon = -\mathbf{H}^{-1} \nabla f(\theta)$ , and it comes to:

$$\theta_{t+1} = \theta_t - \eta \mathbf{H}^{-1} \nabla f(\theta_t),$$

where  $\eta$  is the step size.

### 2.3 Gradient Descent (GD)

The gradient descent method is the first-order approximation of Newton method. **Full GD:**

- Using full datasets for gradient descent, 1.  $\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t) = \theta_t - \eta g_t$ . Here and below,  $g_t$  is defined the gradient of full data/mini batch data on  $\theta_t$ .
- Convergence  $\mathcal{O}(\frac{1}{T})$ . Here the convergence is defined by  $f(x^T) - f^*$

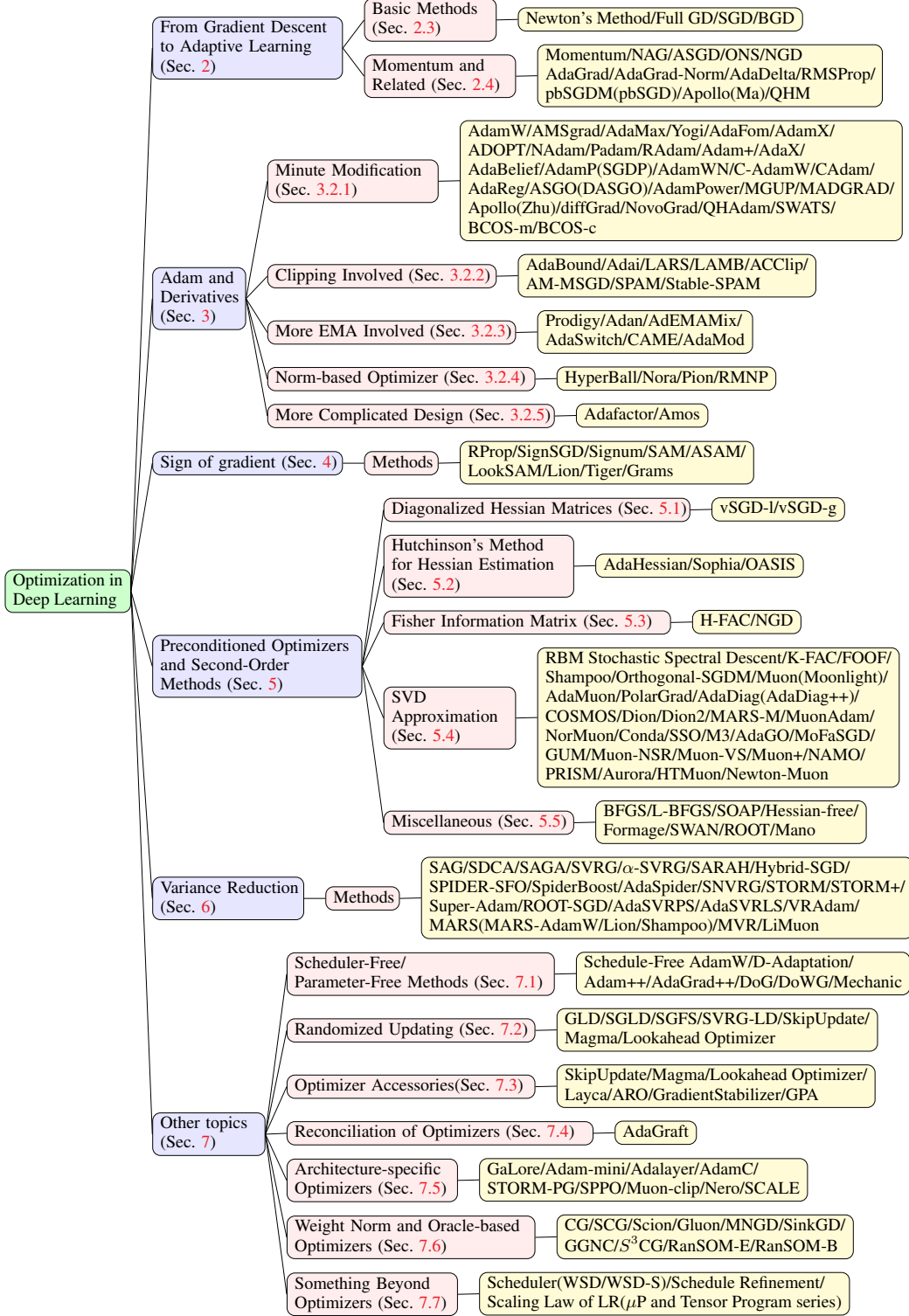


Figure 2: Summary of all the optimizers covered in this paper.

**Stochastic GD (SGD):** Using one sample per step, convergence  $O(\frac{1}{\sqrt{T}})$

**Batch GD (BGD):** Using small batch (size=b) per steps, convergence  $O(\frac{1}{\sqrt{bT}} + \frac{1}{T})$

## 2.4 Momentum and Related

**Momentum** (Grum, 2023)  $m_{t+1} = \gamma m_t + \eta \nabla_{\theta} J(\theta_t)$ ,  $\theta_{t+1} = \theta_t - m_{t+1}$

**Nesterov's Accelerated Gradient (NAG)** (Nesterov, 1983)  $m_{t+1} = \gamma m_t + \eta \nabla_{\theta} J(\theta_t - \gamma m_t)$ ,  $\theta_{t+1} = \theta_t - m_{t+1}$

**ASGD** (Polyak & Juditsky, 1992)  $\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{t+1} \sum_{i=1}^t g_i$

**Online Newton Step (ONS)** (Hazan et al., 2007)  $r_t = r_{t-1} + g_t^2$ ,  $\theta_{t+1} = \theta_t - \frac{\eta}{r_t + \epsilon} g_t$

**Normalized Gradient Descent (NGD)** (Hazan et al., 2015)  $\theta_{t+1} = \theta_t - \eta \frac{g_t}{\|g_t\|}$ .

**AdaGrad** (Duchi et al., 2011)  $r_t = r_{t-1} + g_t^2$ ,  $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t} + \epsilon} g_t$

**AdaGrad-Norm** (Ward et al., 2020)  $r_t = r_{t-1} + \|g_t\|^2$ ,  $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t} + \epsilon} g_t$

**AdaDelta** (Zeiler, 2012)  $v_t = \rho v_{t-1} + (1 - \rho) g_t^2$ ,  $\theta_t = \theta_{t-1} - \eta \frac{\sqrt{u_{t-1}}}{\sqrt{v_t} + \epsilon} g_t$ ,  $u_t = \rho u_{t-1} + (1 - \rho) \Delta \theta_t^2$ , where  $\Delta \theta_t = \theta_t - \theta_{t-1}$

**RMSProp** (Tieleman & Hinton, 2012)  $v_t = \rho v_{t-1} + (1 - \rho) g_t^2$  (EMA (Exponential Moving Average) of the squared gradient),  $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t$

**pbSGDM** (Zhou et al., 2020a)  $v_{t+1} = \beta v_t - \eta \text{sign}(g_t) |g_t|^\gamma$ ,  $\theta_{t+1} = \theta_t + v_{t+1}$ , when  $\beta = 0$ , it reduces to **pbSGD**, and when  $\gamma = 0$ , it reduces to **SGD**.

**Apollo(Ma)** (Ma, 2020)  $m_{t+1} = \frac{\beta(1-\beta^t)}{1-\beta^{t+1}} m_t + \frac{1-\beta}{1-\beta^{t+1}} g_t$ ,  $B_{t+1} = B_t - \frac{d_t^\top (m_{t+1} - m_t) + d_t^\top B_t d_t}{\|d_t\|_4 + \epsilon}$ ,  $D_{t+1} = \max(B_{t+1}, 0.01)$ ,  $d_{t+1} = D_{t+1}^{-1} m_{t+1} + \lambda \theta_t$ ,  $\theta_{t+1} = \theta_t - \eta d_{t+1}$ .

**QHM** (Ma & Yarats, 2018)  $v_{t+1} = \beta v_t + (1 - \beta) g_t$ ,  $\theta_{t+1} = \theta_t - \eta[(1 - \gamma) g_t + \gamma v_{t+1}]$ .

## 3 Adam and Derivatives

**Adam** (Kingma & Ba, 2015)

- $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$  (EMA of gradient, use  $m_t$  below if have same expression)
- $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$  (EMA of squared gradient, use  $v_t$  below if have same expression)
- $\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$ , where  $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$ ,  $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$  (Use  $\hat{m}_t$  or  $\hat{v}_t$  below if have same expression)

### 3.1 Other forms for Adam

#### 3.1.1 Hessian Matrices

$\theta_{t+1} = \arg \min_{\theta} \{ \eta \langle m_t, \theta \rangle + \frac{1}{2} \|\theta - \theta_t\|_{\mathbf{H}_t}^2 \}$ . In closed form:  $\theta_{t+1} = \theta_t - \eta \mathbf{H}_t^{-1} m_t$ . (In some works,  $P_t = \mathbf{H}_t^{-1} \in \mathbb{S}_{++}$  is the *preconditioning matrix* or *preconditioner* (Lau et al., 2025).)

- $\mathbf{H}_t = \sqrt{\text{diag}(v_t)} \cdot \frac{1 - \beta_1^t}{\sqrt{1 - \beta_2^t}}$  for AdamW;
- $\mathbf{H}_t = \sqrt{\text{diag}(m_t^2)}$  for Lion;
- $\mathbf{H}_t = (\sum_{\tau=1}^t \mathbf{G}_t \mathbf{G}_t^\top)^{1/4} \otimes (\sum_{\tau=1}^t \mathbf{G}_t^\top \mathbf{G}_t)^{1/4}$  for Shampoo.

#### 3.1.2 Matrix form of $v_t$

$\theta_{t+1} = \prod_{\mathcal{F}, \sqrt{V_t}} (\theta_t - \eta_t m_t / \sqrt{\hat{v}_t})$ , where  $\theta \in \mathcal{F}$ ,  $V_t = \text{diag}(\hat{v}_t)$

### 3.2 Derivatives of Adam

In the following algorithms, if not specified,  $m_t$ ,  $v_t$ ,  $\hat{m}_t$ ,  $\hat{v}_t$  and the updating rule are the same with Adam, or AdamW with decoupled weight decay.

### 3.2.1 Minute Modification

**AdamW** (Loshchilov & Hutter, 2019)  $\theta_{t+1} = \theta_t - \eta(\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda\theta_t)$ , where  $\lambda$  is called decoupled weight decay (hyper-parameter). And The decoupled weight decay can be applied to the algorithms below.

**AMSgrad** (Reddi et al., 2018)  $\tilde{v}_t = \max(\tilde{v}_{t-1}, v_t)$ ,  $\hat{v}_t = \frac{\tilde{v}_t}{1 - \beta_2^t}$ .

**AdaMax** (Loshchilov & Hutter, 2019)  $u_t = \max(\beta_2 u_{t-1}, |g_t|)$ ,  $\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{u_t}$ .

**Yogi** (Zaheer et al., 2018)  $v_t = v_{t-1} - (1 - \beta_2)\text{sign}(v_{t-1} - g_t^2)g_t^2$ .

**AdaFom** (Chen et al., 2018)  $v_t = (1 - 1/t)v_{t-1} + (1/t)g_t^2$ ,  $\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t}}$ .

**AdamX** (Tran et al., 2019)  $\hat{m}_t = m_t$ ,  $\hat{v}_t = \max\{\frac{(1-\beta_{1,t})^2}{(1-\beta_{1,t-1})^2}\hat{v}_{t-1}, v_t\}$ .

**ADOPT** (Taniguchi et al., 2024)  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)\frac{g_t}{\max\sqrt{v_t}, \epsilon}$ ,  $\theta_{t+1} = \theta_t - \eta m_t$ .

**NAdam** (Dozat, 2016)  $\theta_{t+1} = \theta_t - \eta \frac{1}{\sqrt{\hat{v}_t + \epsilon}}(\beta_1 \hat{m}_t + \frac{1 - \beta_1}{1 - \beta_1^t} g_t)$ , by taking  $\psi = 0$  in formula below

- In PyTorch implementation,  $\theta_{t+1} = \theta_t - \eta \frac{1}{\sqrt{\hat{v}_t + \epsilon}}(\mu_{t+1} \frac{m_t}{1 - \prod_{i=1}^{t+1} \mu_i} + \frac{(1 - \mu_t)}{1 - \prod_{i=1}^t \mu_i} g_t)$ , where  $\mu_t = \beta_1(1 - \frac{0.96^t \psi}{2})$ .

**Padam** (Chen & Gu, 2018)  $\tilde{v}_t = \max(\tilde{v}_{t-1}, v_t)$ ,  $\theta_{t+1} = \theta_t - \eta \frac{m_t}{\tilde{v}_t}$ . And the output is chosen from  $\{\theta_t\}$  with  $P(\theta_{out} = \theta_t) = \frac{\eta_{t-1}}{\sum_{i=1}^{T-1} \eta_i}$ .

**RAdam** (Liu et al., 2020a)  $\rho_\infty = \frac{2}{1 - \beta_2} - 1$ ,  $\rho_t = \rho_\infty - \frac{2t\beta_2^t}{1 - \beta_2^t}$ . If  $\rho_t > 4$ ,  $l_t = \sqrt{(1 - \beta_2^t)/v_t}$ ,  $r_t = \sqrt{\frac{(\rho_t - 4)(\rho_t - 2)\rho_\infty}{(\rho_\infty - 4)(\rho_\infty - 2)\rho_t}}$ ,  $\theta_{t+1} = \theta_t - \eta r_t \hat{m}_t l_t$ ; otherwise  $\theta_{t+1} = \theta_t - \eta \hat{m}_t$ .

**Adam<sup>+</sup>** (Liu et al., 2020b)  $\eta_t = \frac{\alpha\beta^\gamma}{\max(\|z_t\|^{1/2}, \epsilon_0)}$ ,  $\theta_{t+1} = \theta_t - \eta z_t$ ,  $z_{t+1} = (1 - \beta)z_t + \beta \nabla_\theta J((1 - \frac{1}{\beta})\theta_t + \frac{1}{\beta}\theta_{t+1})$ , where  $a$  in the original paper is replaced with  $\gamma$  for clarity.

**AdaX** (Li et al., 2020)  $v_t = (1 + \beta_2)v_{t-1} + \beta_2 g_t^2$ ,  $\hat{v}_t = \frac{v_t}{(1 + \beta_2)^t - 1}$ .

**AdaBelief** (Zhuang et al., 2020)  $v_t = \beta_2 v_{t-1} + (1 - \beta_2)(g_t - m_t)^2 + \epsilon$ .

**AdamP** (Heo et al., 2021)  $p_t = \frac{m_t}{\sqrt{\hat{v}_t + \epsilon}}$ . If  $\cos(\theta_t, g_t) < \delta/\sqrt{\dim(\theta_t)}$ ,  $q_t = p_t - \frac{(\theta_t \cdot p_t)\theta_t}{\|\theta_t\|_2^2}$ , else  $q_t = p_t$ .  $\theta_{t+1} = \theta_t - \eta q_t$ .

- **SGDP** Similar to AdamP, but  $p_t = \beta p_{t-1} + g_t$ .

**AdamWN** (Loshchilov, 2023)  $\hat{\theta}_t = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$ ,  $\theta_{t+1} = \hat{\theta}_t - k_t(1 - \frac{r_t \|\theta_0\|}{\|\hat{\theta}_t\|})\hat{\theta}_t$ ,  $k_t \in [0, 1]$ ,  $r_t \|\theta_0\|$  is the target weight norm for  $\theta_t$ .

**C-AdamW** (Liang et al., 2024)  $u_t = \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$ ,  $\phi_t = \mathbf{1}_{u_t \odot g_t \geq 0}$ ,  $\bar{\eta}_t = \eta \frac{d}{\|\phi_t\|_0 + 1}$ ,  $\theta_{t+1} = \theta_t - \bar{\eta}_t(\phi_t \odot u_t + \lambda\theta_t)$ .

**CAdam** (Wang et al., 2024)  $\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \odot \mathbf{1}_{m_t \odot g_t > 0}$ .

**AdaReg** (Gupta et al., 2017)  $r_t = r_{t-1} + g_t g_t^\top$ ,  $H_t = \arg \min_H \{\text{Tr}(r_t^\top H) + \Phi(H)\}$ ,  $\theta_{t+1} = \theta_t - H_t g_t$ .

- $\Phi(H) = \text{Tr}(H^{-1})$  ( $H_t = (\sum_{s=1}^t g_s g_s^\top)^{-1/2}$ ) for AdaGrad (Duchi et al., 2011);
- $\Phi(H) = -\log |H|$  ( $H_t = (\sum_{s=1}^t g_s g_s^\top)^{-1}$ ) for ONS (Hazan et al., 2007).

**ASGO** (An et al., 2025) For 2-dimension  $\theta_t, g_t$ ,  $r_t = r_{t-1} + g_t g_t^\top$ ,  $\Lambda_t = r_t^{1/2} + \epsilon I_m$ ,  $\theta_{t+1} = \theta_t - \eta \Lambda_t^{-1} g_t$ .

- **DASGO** For 2-dimension  $\theta_t, g_t, v_t = \beta_2 v_{t-1} + (1 - \beta_2)\text{diag}(g_t^\top g_t)$ ,  $\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t + \epsilon}}$ .

**AdamPower** Use powered gradient (Wang et al., 2025b)  $\tilde{g}_t = |g_t|^p \text{sign}(g_t)$  to replace the gradient in AdamW.

**MGUP-AdamW** (Chang & Yuan, 2025)  $\eta_t = \eta \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}$ ,  $u_t = \frac{m_t}{\sqrt{v_t+\epsilon}}$ ,  $\phi_t = \text{MGUP}(u_t \odot g_t)$ ,  $\theta_{t+1} = \theta_t - \eta_t(\phi_t \odot u_t + \lambda\theta_t)$ , where MGUP outputs  $\phi_{t,i} = 1/\tau$  for  $i \in \mathcal{I}_{\text{topK}}$  with  $\mathcal{I}_{\text{topK}}$  as the index set of the largest K elements of  $u_t \cdot g_t$  with  $K = \lfloor \tau \cdot d \rfloor$ , otherwise  $\phi_{t,i} = \tau$ .

**MADGRAD** (Defazio & Jelassi, 2022)  $\lambda_t = \eta\sqrt{t+1}$ ,  $m_t = m_{t-1} + \lambda_t g_t$ ,  $v_t = v_{t-1} + \lambda_t g_t^2$ ,  $z_{t+1} = \theta_0 - \frac{s_t}{\sqrt[3]{v_t+\epsilon}}$ ,  $\theta_{t+1} = (1-\beta)\theta_t + \beta z_{t+1}$ .

**Apollo(Zhu)** (Zhu et al., 2025) (Only for 2D parameters  $\theta \in \mathbb{R}^{m \times n}$ ) For every T step, sample  $P_t \sim \mathcal{N}(0, 1/r)$ ,  $R_t = P_t g_t$ , use  $R_t$  to get  $m_t$ ,  $v_t$  in Adam,  $\tilde{R}_t = \frac{m_t}{\sqrt{v_t+\epsilon}}$ ,  $s_{t,i} = \frac{\|\tilde{R}_t[:,i]\|_2}{\|R_t[:,i]\|_2}$ ,  $S_t = \text{diag}(s_{t,0}, \dots, s_{t,m-1})$ , (if Apollo-mini,  $S_t = \text{diag}(\frac{\|\tilde{R}_t\|_2}{\|R_t\|_2})$ ).  $\theta_{t+1} = \theta_t + \eta(\alpha g_t S_t - \lambda\theta_t)$ .

**diffGrad** (Dubey et al., 2019)  $\theta_{t+1} = \theta_t - \frac{\eta m_t}{(1+e^{-|g_t-g_{t-1}|})(\sqrt{v_t+\epsilon})}$ .

**NovoGrad** (Ginsburg et al., 2019)  $v_t = \beta_2 v_{t-1} + (1-\beta_2)\|g_t\|_2^2$ ,  $m_t = \beta_1 m_{t-1} + (\frac{g_t}{\sqrt{v_t+\epsilon}} + \lambda\theta_t)$ ,  $\theta_{t+1} = \theta_t - \eta m_t$ .

**QHAdam** (Ma & Yarats, 2018)  $\theta_{t+1} = \theta_t - \eta \left[ \frac{(1-\gamma_1)g_t + \gamma_1 \hat{m}_t}{\sqrt{(1-\gamma_2)g_t^2 + \gamma_2 \hat{v}_t + \epsilon}} \right]$ .

**SWATS** (Keskar & Socher, 2017) Update with Adam for initial steps and  $\gamma_t = \frac{p_t^\top p_t}{-p_t^\top g_t}$ ,  $\lambda_t = \beta_2 \lambda_{t-1} + (1-\beta_2 \gamma_t)$  when  $p_t^\top g_t \neq 0$ , where  $p_t = -\eta \frac{\hat{m}_t}{\sqrt{v_t+\epsilon}}$ , until  $|\frac{\lambda_t}{1-\beta_2^t} - \gamma_t| < \epsilon$ , then  $v_t = 0$ ,  $\Lambda = \frac{\lambda_t}{1-\beta_2^t}$ , and after that, update with SGD:  $v_t = \beta_1 v_{t-1} + g_t$ ,  $\theta_{t+1} = \theta_t - (1-\beta_1)\Lambda v_t$ .

**BCOS-m** (Jiang & Xiao, 2025)  $v_t = \beta_2 v_{t-1} + (1-\beta_2)m_t^2$ ,  $\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t+\epsilon}}$ . (BCOSW-m:  $\theta_{t+1} = \theta_t - \eta(\frac{m_t}{\sqrt{v_t+\epsilon}} + \lambda\theta_t)$ )

**BCOS-c** (Jiang & Xiao, 2025)  $v_t = (1-(1-\beta_1)^2)m_{t-1}^2 + (1-\beta_1)^2 g_t^2$ ,  $\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t+\epsilon}}$ . (BCOSW-c:  $\theta_{t+1} = \theta_t - \eta(\frac{m_t}{\sqrt{v_t+\epsilon}} + \lambda\theta_t)$ )

### (Potential) Unified Analysis of Adaptive Optimizers

- General Adaptive Optimization algorithm: For cumulative variante  $m_t = m_{t-1} + g_t g_t^\top$ ; for EMA variant  $m_t = \beta m_{t-1} + (1-\beta)g_t g_t^\top$ ; for weighted variant  $m_t = \beta m_{t-1} + g_t g_t^\top$ .  $V_t = \arg \min_{\mathbf{H} \in \mathcal{H}} \langle m_t + \epsilon I_d, \mathbf{H}^{-1} \rangle + \text{tr}(\mathbf{H})$  (closed form  $V_t = (m_t + \epsilon I_d)^{1/2}$ ),  $\theta_{t+1} = \theta_t - \eta V_t^{-1} g_t$ .
- Accelerated Adaptive algorithm: same as the cumulative variant above, but  $g_t = \alpha_t^{-2} \nabla f(\alpha_t \theta_t + (1-\alpha_t)\bar{\theta}_t)$ ,  $\bar{\theta}_{t+1} = \alpha_t \theta_{t+1} + (1-\alpha_t)\bar{\theta}_t$  and return  $\bar{\theta}_T$ .
- Accelerated Adaptive algorithm with Projection: same as above, but  $\tilde{\theta}_t = \theta_t - \eta V_t^{-1} g_t$ ,  $\theta_{t+1} = \arg \min_{\|\theta\|_{\mathcal{H}} \leq D} \|\theta - \tilde{\theta}_t\|_{V_t}^2$ ,  $\bar{\theta}_{t+1} = \alpha_t \tilde{\theta}_t + (1-\alpha_t)\bar{\theta}_t$ .
- Normalized steepest descent with momentum:  $m_t = \beta m_{t-1} + (1-\beta)g_t$ ,  $u_t = \arg \max_{\|u\| \leq 1} \langle m_t, u \rangle$ ,  $\theta_{t+1} = \theta_t - \eta u_t$ .

### 3.2.2 Clipping Involved

**AdaBound** (Luo et al., 2019)  $\eta_t = \text{Clip}(\alpha/\sqrt{\text{diag}(v_t)}, \eta_l, \eta_u)/\sqrt{t}$ , where  $\eta_l$  and  $\eta_u$  are lower bound and upper bound of learning rates,  $\theta_{t+1} = \theta_t - \eta_t m_t$ .

**Adai** (Xie et al., 2022)  $\bar{v}_t = \text{mean}(\hat{v}_t)$ ,  $\beta_{1,t} = \text{Clip}(1 - \frac{\beta_0}{\bar{v}_t} \hat{v}_t, 0, 1 - \epsilon)$ ,  $\hat{m}_t = \frac{m_t}{1 - \Pi_{s=1}^t \beta_{1,s}}$ ,  $\theta_{t+1} = \theta_t - \eta \hat{m}_t$ .

**LARS** (You et al., 2017)  $m_t = \beta_1 m_{t-1} + (1-\beta_1)(g_t + \lambda\theta_t)$ ,  $\theta_{t+1} = \theta_t - \eta \frac{\text{Clip}(\|\theta_t\|, \gamma_l, \gamma_u)}{\|\theta_t\| + \epsilon} m_t$ , where  $\text{Clip}(x, \gamma_l, \gamma_u) = \min(\max(x, \gamma_l), \gamma_u)$ ,  $\gamma_l$  is default to 0 if only 1 bound given

**LAMB** (You et al., 2020)  $r_t = \frac{\hat{m}_t}{\sqrt{\hat{v}_t+\epsilon}}$ ,  $\theta_{t+1} = \theta_t - \eta \frac{\text{Clip}(\|\theta_t\|, \gamma_l, \gamma_u)}{\|r_t + \lambda\theta_t\| + \epsilon} (r_t + \lambda\theta_t)$

**ACClip** (Zhang et al., 2020)  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ ,  $v_t^\alpha = \beta_2 v_t^\alpha + (1 - \beta_2)|g_t|^\alpha$ ,  $\theta_t = \theta_{t-1} - \eta m_t \cdot \min\{\frac{v_t}{|m_t|+\epsilon}, 1\}$ .

**AM-MSGD** (Topollai & Choromanska, 2025)  $\beta_t = \text{Clip}(\frac{(\lambda+1)g_t^\top d_t - \langle d_t - g_t, g_t + \lambda d_t \rangle}{\|d_t - g_t\|_2^2}, 0, \beta_{\max, t})$ ,  $d_{t+1} = \frac{\beta_t + \lambda}{1 + \lambda} d_t + \frac{1 - \beta_t}{1 + \lambda} g_t$ ,  $\theta_{t+1} = \theta_t - \eta d_{t+1}$ .

**SPAM** (Huang et al., 2025c) (Based on Adam) For each  $\Delta T$  iterations, reset  $m_t$  and  $v_t$  as 0, and sample 0-1 variant  $M$  with probability  $d$ .  $g_t \leftarrow \text{Clip}(g_t, -\sqrt{\tau v_t}, \sqrt{\tau v_t})$ ,  $\theta_{t+1} = \theta_t - \eta r_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$ , where  $r_t = 1 - \text{CosineAnnealing}(\text{Mod}(t, \Delta T), N)$  is the warmup scale.

**Stable-SPAM** (Huang et al., 2025b) (Based on Adam)  $T_t = T_{t-1}\tau + (1 - \tau) \max(|g_t|)$ ,  $g_t \leftarrow \text{Clip}(g_t, -\frac{T_t}{1 - \tau}, \frac{T_t}{1 - \tau})$ .  $m'_t = \gamma_1 m'_{t-1} + (1 - \gamma_1)\|g_t\|_2$ ,  $v'_t = \gamma_2 v'_{t-1} + (1 - \gamma_2)\|g_t\|_2^2$ ,  $\hat{m}'_t = \frac{m'_t}{1 - \gamma_1}$ ,  $\hat{v}'_t = \frac{v'_t}{1 - \gamma_2}$ ,  $g_t \leftarrow \frac{g_t}{\|g_t\|_2} \frac{\hat{m}'_t}{\sqrt{\hat{v}'_t + \epsilon}}$ . For each  $\Delta T$  iterations, reset  $m_t$  and  $v_t$  as 0, then optimize with Adam with  $g_t$ .

### 3.2.3 More EMA Involved

Some optimizers introduce more EMA to stabilize training or add new features to the optimizers. However, such increment will also require more memory, especially for training frameworks like Megatron (Shoeybi et al., 2019).

**Prodigy** (Bernstein & Newhouse, 2024)  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)\eta_t g_t$ ,  $v_t = \beta_2 v_{t-1} + (1 - \beta_2)\eta_t^2 g_t^2$ ,  $r_t = \sqrt{\beta_2} r_{t-1} + (1 - \sqrt{\beta_2})\eta_t^2 g_t^\top (\theta_0 - \theta_t)$ ,  $s_t = \sqrt{\beta_2} s_{t-1} + (1 - \sqrt{\beta_2})\eta_t^2 g_t$ ,  $\eta_{t+1} = \max(\eta_t, \frac{r_t}{\|s_t\|_1})$ ,  $\theta_{t+1} = \theta_t - \eta_t \frac{m_t}{\sqrt{v_t}}$ .

**Adan** (Xie et al., 2024)  $v_t = (1 - \beta_2)v_{t-1} + \beta_2(g_t - g_{t-1})$ ,  $n_t = (1 - \beta_3)n_{t-1} + \beta_3[g_t + (1 - \beta_2)(g_t - g_{t-1})]^2$ ,  $\theta_t = \frac{1}{1 + \lambda\eta}[\theta - \frac{\eta}{\sqrt{n_t + \epsilon}} \circ (m_t + (1 - \beta_2)v_t)]$ .

**AdEMAMix** (Pagliardini et al., 2024)  $r_t = \beta_3 r_{t-1} + (1 - \beta_3)g_t$ ,  $\theta_t = \theta_{t-1} - \eta(\frac{\hat{m}_t + \alpha_t r_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \theta_{t-1})$ .

**AdaSwitch** (Zou et al., 2025) (Based on AdamW)  $d_t = \beta_1 d_{t-1} + g_t$  (here  $\beta_1$  is the first momentum factor in AdamW)  $\theta_{t+1} = \theta_t - \eta_1 d_t - \eta_2 \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$ .

**CAME** (Luo et al., 2023)  $r_t = \beta_2 r_{t-1} + (1 - \beta_2)(g_t^2 + \epsilon_1)$ ,  $c_t = \beta_2 c_{t-1} + (1 - \beta_2)(g_t^2 + \epsilon_1)$ ,  $v_t = \frac{r_t c_t}{\sum r_t}$ ,  $u_t = g_t / \sqrt{v_t}$ ,  $\hat{u}_t = \frac{u_t}{\max(1, \text{RMS}(u_t)/d)}$ ,  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)\hat{u}_t$ ,  $U_t = (\hat{u}_t - m_t)^2$ ,  $R_t = \beta_3 R_{t-1} + (1 - \beta_3)(U_t + \epsilon_2)$ ,  $C_t = \beta_3 C_{t-1} + (1 - \beta_3)(U_t + \epsilon_2)$ ,  $S_t = \frac{R_t C_t}{\sum R_t}$ ,  $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{S_t}} m_t$ .

**AdaMod** (Ding et al., 2019) (Based on Adam)  $\eta_t = \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}}$ ,  $s_t = \beta_3 s_{t-1} + (1 - \beta_3)\eta_t$ ,  $\hat{\eta}_t = \min(\eta_t, s_t)$ ,  $\theta_{t+1} = \theta_t - \hat{\eta}_t \hat{m}_t$ .

### 3.2.4 Norm-based Optimizer

**HyperBall** (Wen et al., 2025)  $R = \|\theta_0\|_F$ ,  $\text{Normalize}(\mathbf{x}) = \mathbf{x} / \|\mathbf{x}\|_F$ ,  $\theta_{t+1} = R \cdot \text{Normalize}(\theta_t - \eta R \cdot \text{Normalize}(u_t))$ , where  $u_t$  is Adam or Muon update.

**Nora** (Yuan et al., 2026) (Only for 2D parameters  $\theta \in \mathbb{R}^{m \times n}$ )  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $r_{t,i} = m_{t,i} - \frac{\langle v_{t,i}, \theta_{t,i} \rangle}{\|\theta_{t,i}\|_2^2} \theta_{t,i}$ ,  $\theta_{t+1} = \theta_t - \eta_t (\text{diag}(r_t r_t^\top)^{-1/2} r_t + \lambda \theta_t)$ .

**Pion** (Shi et al., 2026) (Only for 2D parameters  $\theta \in \mathbb{R}^{m \times n}$ ) Define  $E(A, \alpha) = I + \eta \alpha A + \frac{1}{2}(\eta \alpha A)^2$ .  $g_t^{\text{in}} = \theta_t^\top g_t - g_t^\top \theta_t$ ,  $g_t^{\text{out}} = g_t \theta_t^\top - \theta_t g_t^\top$ .  $m_t^{\text{in}} = \beta_1 m_{t-1}^{\text{in}} + (1 - \beta_1)g_t^{\text{in}}$ ,  $m_t^{\text{out}} = \beta_1 m_{t-1}^{\text{out}} + (1 - \beta_1)g_t^{\text{out}}$ ;  $v_t^{\text{in}} = \beta_2 v_{t-1}^{\text{in}} + (1 - \beta_2)(g_t^{\text{in}} \odot g_t^{\text{in}})$ ,  $v_t^{\text{out}} = \beta_2 v_{t-1}^{\text{out}} + (1 - \beta_2)(g_t^{\text{out}} \odot g_t^{\text{out}})$ ;  $A_t^{\text{in}} = -\frac{m_t^{\text{in}}}{\sqrt{v_t^{\text{in}} + \epsilon}}$ ,  $A_t^{\text{out}} = -\frac{m_t^{\text{out}}}{\sqrt{v_t^{\text{out}} + \epsilon}}$ . If alternative update is used, for even  $t$ ,  $\alpha_t = \frac{c\sqrt{mn}}{\|A_t^{\text{out}} \theta_t\|_F + \epsilon}$ ,  $\theta_{t+1} = E(A_t^{\text{out}}, \alpha_t) \theta_t$ ; otherwise  $\alpha_t = \frac{c\sqrt{mn}}{\|\theta_t A_t^{\text{in}}\|_F + \epsilon}$ ,  $\theta_{t+1} = \theta_t E(A_t^{\text{in}}, \alpha_t)$ . If alternative update is not used,  $\alpha_t = \frac{c\sqrt{mn}}{\|A_t^{\text{out}} \theta_t + \theta_t A_t^{\text{in}}\|_F + \epsilon}$ ,  $\theta_{t+1} = E(A_t^{\text{out}}, \alpha_t) \theta_t E(A_t^{\text{in}}, \alpha_t)$ .



**RMNP** (Deng et al., 2026)  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $d_{t,i} = \frac{m_{t,i}}{\|m_{t,i}\|_2}$  (Row-norm) or  $d_t = \text{diag}((m_t m_t^\top)^{-1/2})m_t$ ,  $\theta_{t+1} = \theta_t - \eta d_t$ .

### 3.2.5 More Complicated Design

**Adafactor** (Shazeer & Stern, 2018)  $u_t = \frac{g_t}{\sqrt{\hat{v}_t}}$ ,  $\hat{u}_t = \frac{u_t}{\max(1, \text{RMS}(u_t)/d)}$ ,  $\theta_{t+1} = \theta_t - \eta_t \hat{u}_t$ , where  $\alpha_t = \eta \cdot \max(\epsilon_2, \text{RMS}(\theta_t))$ ,  $\beta_{2,t} = 1 - t^\tau$ , and

- For weight vector  $\theta_t \in \mathbb{R}^n$ ,  $\hat{v}_t = \beta_{2,t} \hat{v}_{t-1} + (1 - \beta_{2,t})(g_t^2 + \epsilon_1 1_n)$ ;
- For weight matrix  $\theta_t \in \mathbb{R}^{m \times n}$ ,  $r_t = \beta_{2,t} r_{t-1} + (1 - \beta_{2,t})(g_t^2 + \epsilon_1 1_m 1_n^\top) 1_n$ ,  $v_t = \beta_{2,t} v_{t-1} + (1 - \beta_{2,t}) 1_m^\top (g_t^2 + \epsilon_1 1_m 1_n^\top)$ ,  $\hat{v}_t = r_t v_t / 1_m^\top r_t$ .

**Amos** (Tian & Parikh, 2022)  $c_t = (1 + \frac{1}{4} \sqrt{\eta} b_t)^{-1/2}$ ,  $d_t = (1 + \frac{1}{4} \sqrt{\eta} \tilde{\eta} b_t)^{-1}$ , with  $\tilde{\eta}$  the expected scale for model weights  $\theta$ , (Optional)  $\tilde{g}_t = \frac{\chi}{\max(\chi, \|g_t\|)} g_t$ ,  $v_t = \beta v_{t-1} + (1 - \beta) M_2(\tilde{g}_t)^2$ , where  $M_2(a) := \sqrt{\frac{1}{k} \sum_{i=1}^k a_i^2}$  is the quadratic mean of the entries.  $\hat{v}_t = \frac{v_t}{1 - \beta t}$ ,  $\gamma_t = c_t \frac{\eta^2}{v_t} M_2(g_t)^2$ ,  $\delta_t = d_t (\frac{\eta \tilde{\eta}}{\sqrt{\hat{v}_t}} g_t + \frac{\gamma_t}{2} \theta_t)$ .  $b_{t+1} = b_t + \gamma_t (1 + b_t)$ , (Optional)  $\tilde{\delta}_t = m_{t+1} = \mu m_t + (1 - \mu) \delta_t$ ,  $\theta_{t+1} = \theta_t - \tilde{\delta}_t$

## 4 Sign of Gradient

**RProp** (Riedmiller & Braun, 1993) for  $0 < \eta^- < 1 < \eta^+$ ,

- If  $g_t \cdot g_{t-1} > 0$ ,  $\eta_t = \min(\eta^+ \eta_{t-1}, \eta_{max})$ ,  $\theta_{t+1} = \theta_t - \eta_t \text{Sign}(g_t)$
- If  $g_t \cdot g_{t-1} < 0$ ,  $\eta_t = \max(\eta^- \eta_{t-1}, \eta_{min})$ ,  $\theta_{t+1} = \theta_t - \Delta \theta_t$ , where  $\Delta \theta_t = \theta_t - \theta_{t-1}$
- Otherwise  $\eta_t = 1$ ,  $\theta_{t+1} = \theta_t - \eta_t \text{Sign}(g_t)$

**SignSGD** (Bernstein et al., 2018)  $\theta_{t+1} = \theta_t - \eta \text{Sign}(g_t)$ .

**Signum** (Bernstein et al., 2018)  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $\theta_{t+1} = \theta_t - \eta \text{Sign}(m_t)$ .

**Sharpness-Aware Minimization (SAM)** (Foret et al., 2021)  $\hat{\epsilon}_t = \rho \text{Sign}(g_t) |g_t|^{q-1} / \|g_t\|_q^{q/p}$ ,  $\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta_t + \hat{\epsilon}_t)$ .

**ASAM** (Kwon et al., 2021)  $\theta_{t+1} = \theta_t - \alpha (\nabla_\theta J(\theta_t + \rho \frac{T_\theta^2 g_t}{\|T_\theta^2 g_t\|_2}) + \lambda \theta_t)^\dagger$ , where  $T_\theta$  is some invertible linear operator such as  $T_\theta = \text{diag}(|\theta|)$ .

**LookSAM** (Liu et al., 2022) If  $t \bmod k = 0$ ,  $\bar{g}_t = \nabla_\theta J(\theta + \rho g_t / \|g_t\|)$ ,  $\tilde{g}_t = \bar{g}_t - \|\bar{g}_t\| \frac{(g_t \cdot \bar{g}_t) g_t}{\|g_t\|^2 \|\bar{g}_t\|}$ ; else  $\bar{g}_t = g_t + \alpha \frac{\|g_t\|}{\|\bar{g}_{t-1}\|} \tilde{g}_{t-1}$ ,  $\tilde{g}_t = \tilde{g}_{t-1}$ .  $\theta_{t+1} = \theta_t - \eta \tilde{g}_t$ .

**Lion** (Chen et al., 2023)  $c_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ ,  $m_t = \beta_2 m_{t-1} + (1 - \beta_2)g_t$ ,  $\theta_{t+1} = \theta_t - \eta (\text{Sign}(c_t) + \lambda \theta_t)$

- **Tiger** (Su, 2023) (A special case of Lion when  $\beta_1 = \beta_2 = \beta$ )  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $\theta_{t+1} = \theta_t - \eta (\text{Sign}(m_t) + \lambda \theta_t)$ . For bias and normalization parameters,  $\eta_i = 0.5\eta$ ,  $\lambda = 0$ , otherwise  $\eta_i = \eta \times \text{RMS}(\theta_{t,i})$ ,  $\lambda = \text{Constant} > 0$

**Grams** (Cao et al., 2024) (Based on AdamW)  $\theta_{t+1} = \theta_t - \eta (\text{Sign}(g_t) \circ \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \theta_t)$ .

## 5 Preconditioned Optimizers and Second-Order Methods

Preconditioned approaches usually use a preconditioner  $\mathbf{H}_t$  to adjust the updates by involving the inter-dependencies among model parameters. According to the Newton's Method,

$$\theta_{t+1} = \theta_t - (\nabla_{\theta_t}^2 J(\theta))^{-1} \nabla_{\theta_t} J(\theta_t) = \theta_t - \mathbf{H}_t^{-1} g_t.$$

Therefore, a direct motivation for preconditioned approaches is to directly involve or approximate the second-order information in optimization process. However, direct computation needs  $\mathcal{O}(n^2)$  time, therefore, adequate approximation or estimation is needed.



## 5.1 Diagonalized Hessian Matrices

Usually, diagonalized Hessian matrices and updates are expected. [Becker \(1988\)](#) first use diagonal Hessian as the pre-conditioner by ignoring off-diagonal entries:

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{|\text{diagonal}(\mathbf{H}_t)| + \epsilon},$$

where we use “diagonal” to indicate taking diagonal entries instead of forming diagonal matrix with entries from another vector (denoted as “diag”).

**vSGD-l** ([Schaul et al., 2013](#))  $m_t = (1 - \tau_t^{-1})m_{t-1} + \tau_t^{-1}g_t$ ,  $v_t = (1 - \tau_t^{-1})v_{t-1} + \tau_t^{-1}g_t^2$ ,  $h_t = (1 - \tau_t^{-1})h_{t-1} + \tau_t^{-1}|\text{diagonal}(\mathbf{H}_t)|$ ,  $\eta_t = \frac{m_t^2}{h_t v_t}$ ,  $\tau_{t+1} = (1 - \frac{m_t^2}{v_t})\tau_t + 1$ ,  $\theta_{t+1} = \theta_t - \eta_t g_t$ .

**vSGD-g**, same as vSGD-l, but  $\eta_t = \frac{\sum_i (m_t)_i^2}{h_t \max_i (h_t)_i}$ .

## 5.2 Hutchinson’s Method for Hessian Estimation

To estimate  $\mathbf{H}_t$ , Hutchinson’s method ([Hutchinson, 1989](#)) can be used:  $z \sim \text{Rademacher}(0.5)$ ,  $\frac{\partial g^\top}{\partial \theta} z = \frac{\partial g^\top}{\partial \theta} z + g^\top \frac{\partial z}{\partial \theta} = \frac{\partial g^\top}{\partial \theta} z = \mathbf{H}_t z$ ,  $\mathbf{D} = \text{diag}(\mathbf{H}) = \mathbb{E}[z \odot (\mathbf{H}z)]$ . Such implementation will require gradients on gradients, which requires an additional compute graph and more memory.

**AdaHessian** ([Yao et al., 2021](#)) (Based on Adam)  $v_t = \beta_2 v_{t-1} + (1 - \beta_2) \mathbf{D}_t^2$ ,  $\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{(\sqrt{\hat{v}_t})^{k+\epsilon}}$ , where  $k$  is the Hessian power.

**Sophia** ([Liu et al., 2024b](#))  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ , if  $t \bmod k = 1$  for the size of the step group  $k$ ,  $h_t = \beta_2 h_{t-k} + (1 - \beta_2)\hat{h}_t$ , where  $\hat{h}_t$  is calculated using one of the following Hessian estimators:

- Hutchinson: Draw  $u \sim \mathcal{N}(0, I_d)$ ,  $\hat{h}_t = u \odot \nabla(\langle g_t, u \rangle)$ ;
- Gauss-Newton-Bartlett: Compute logits on mini-batch  $\{l(\theta_t, \xi_{t,b})\}_{b=1}^B$ , sample  $\hat{y}_{t,b} \sim \text{Softmax}(l(\theta_t, \xi_{t,b}))$ ,  $\forall b \in [B]$ .  $\hat{g} = \frac{1}{B} \nabla L(l(\theta_t, \xi_{t,b}), \hat{y}_{t,b})$  with loss function  $l$ ,  $\hat{h} = B \cdot \hat{g} \odot \hat{g}$ ,

Otherwise,  $h_t = h_{t-1}$ .  $\theta_{t+1} = \theta_t - \eta(\text{Clip}(\frac{m_t}{\max\{\gamma_{h_t, \epsilon}\}}, 1) - \lambda \theta_t)$ .

**OASIS** ([Jahani et al.](#))  $v_t = \beta_2 v_{t-1} + (1 - \beta_2) \mathbf{D}_t^2$ ,  $\hat{v}_t = \max\{v_t, \alpha\}$  for a positive truncation value  $\alpha$ ;  $\eta_t = \min\{\sqrt{1 + S_{t-1}}\eta_{t-1}, \frac{\|\theta_t - \theta_{t-1}\|_{\hat{v}_t}}{2\|g_t - g_{t-1}\|_{\hat{v}_t}^*}\}$ , where  $S_0 = +\infty$ ,  $\theta_{t+1} = \theta_t - \eta_t \frac{g_t}{\hat{v}_t}$ ,  $S_t = \frac{\eta_t}{\eta_{t-1}}$ .

## 5.3 Fisher Information Matrix

Another way to estimate Hessian matrix is to compute Fisher information matrix (proposed by [Amari \(1998\)](#) and **H-FAC** ([Martens & Grosse, 2015](#))), since  $\hat{v}_t$  is an approximation to the diagonal of Fisher information matrix ([Pascanu & Bengio, 2014](#)):

$$F = E[\frac{d \log p(y|x, \theta)}{d\theta} (\frac{d \log p(y|x, \theta)}{d\theta})^\top] = E[\mathcal{D}\theta \mathcal{D}\theta^\top].$$

**Natural Gradient Descent (NGD)** ([Amari, 1998](#))  $\theta_{t+1} = \theta_t - \frac{1}{\lambda} F^{-1} g_t$ .

## 5.4 SVD Approximation

Sometimes, to approximate Hessian matrix, SVD is utilized. To approximate this process, they developed Newton-Schulz iteration ([Higham, 2008](#)) methods for computing  $UV^\top$  for  $g = U\Sigma V^\top$ , by setting  $X_0 = g/\|g\|_{l_2 \rightarrow l_2}$  or  $X_0 = g/\|g\|_F$ , where  $\|M\|_{\alpha \rightarrow \beta} := \max_x \frac{\|Mx\|_\beta}{\|x\|_\alpha}$  is the induced operator norm. Then  $UV^\top$  can be approximated by several iteration of  $X_{t+1} = \frac{3}{2}X_t - \frac{1}{2}X_t X_t^\top X_t$ .

- Usually, NewtonSchulz5 is often utilized by applying Newton-Schulz methods for 5 iterations with  $X_{t+1} = aX_t + bX_t X_t^\top X_t + c(X_t X_t^\top)^2 X_t$  for  $(a, b, c) = (3.4445, -4, 7750, 2.0315)$ .

- **Polar Express** (Amsel et al., 2025) (For 5 steps):

$$(a_i, b_i, c_i) = [(8.28721201814563, -23.595886519098837, 17.300387312530933), \\ (4.107059111542203, -2.9478499167379106, 0.5448431082926601), \\ (3.9486908534822946, -2.908902115962949, 0.5518191394370137), \\ (3.3184196573706015, -2.488488024314874, 0.51004894012372), \\ (2.300652019954817, -1.6689039845747493, 0.4188073119525673)].$$

- QR acceleration (He et al., 2025a): for input  $X \in \mathbb{R}^{m \times n}$  low rank  $r$ , draw a Gaussian random matrix  $G \in \mathbb{R}^{n \times r}$ , perform QR decomposition on  $XG$ , then return  $Q$  NewtonSchulz( $Q^\top X$ ).
- *Polar Decomposition* (often denoted as *Orthogonalization*): For any matrix  $M \in \mathbb{R}^{m \times n}$ , it has a polar decomposition  $A = U_p H$  (if  $m \geq n$ ) or  $A = H U_p$  (if  $m < n$ ) for semi-orthogonal matrix  $U_p \in \mathbb{R}^{m \times n}$  and Hermitian matrix  $H \in \mathbb{S}_+^n$  (if  $m \geq n$ ) or  $\mathbb{S}_+^m$  (if  $m < n$ ). For  $U \Sigma V^\top = \text{SVD}(A)$ ,  $U_p = U V^\top$ ,  $H = V \Sigma V^\top$  (if  $m \geq n$ ) or  $H = U \Sigma U^\top$  (if  $m < n$ ). Therefore, in this section, we denote

$$\text{Polar}(A) = U V^\top.$$

In this section, all the algorithms are designed for 2D parameters  $\theta \in \mathbb{R}^{m \times n}$  unless otherwise specified. And use ‘‘Ortho’’ to indicate approximation of SVD.

**RBM Stochastic Spectral Descent** (Carlson et al., 2015) (Only for Restricted Boltzmann Machine  $-E(\mathbf{v}, \mathbf{h}, \theta) = \mathbf{v}^\top \mathbf{c} + \mathbf{h}^\top \mathbf{b} + \mathbf{v}^\top \mathbf{W} \mathbf{h}$ , where  $\theta = \{\mathbf{c}, \mathbf{b}, \mathbf{W}\} \in \{\mathbb{R}^m, \mathbb{R}^n, \mathbb{R}^{m \times n}\}$ ),  $\mathbf{W}_{t+1} = \mathbf{W}_t - \eta \frac{\text{tr}(\Sigma_t)}{mn} \text{Polar}(g_t^{\mathbf{W}})$ ,  $\mathbf{b}_{t+1} = \mathbf{b}_t - \|\mathbf{b}\|_1 \text{Sign}(\mathbf{b})/n$ ,  $\mathbf{c}_{t+1} = \mathbf{c}_t - \|\mathbf{c}\|_1 \text{Sign}(\mathbf{c})/m$ .

**K-FAC** (Martens & Grosse, 2015; Grosse & Martens, 2016) For one layer of neural network  $\mathbf{u}_l = \theta_l \mathbf{h}_{l-1} + \mathbf{b}_l$ ,  $\mathbf{h}_l = \phi(\mathbf{u}_l)$ , for  $G_l = \nabla_{\mathbf{u}_l} J(\theta_l)$ , K-FAC updates  $\theta_l$  by  $L_{l,t} = L_{l,t-1} + G_l G_l^\top$ ,  $R_{l,t} = R_{l,t-1} + \mathbf{h}_{l-1} \mathbf{h}_{l-1}^\top$ ,  $\theta_{l,t+1} = \theta_{l,t} - \eta L_{l,t}^{-1} g_t R_{l,t}^{-1}$ .

**FOOF** (Benzing, 2022) Similar to K-FAC, but  $\Sigma_{l,t} = \beta \Sigma_{l,t-1} + (1 - \beta) G_l G_l^\top$ ,  $L_{l,t} = L_{l,t-1}$  (except for every T steps,  $L_{l,t} = \Sigma_{l,t} + \epsilon \mathbf{I}$ ),  $\theta_{l,t+1} = \theta_{l,t} - \eta L_{l,t}^{-1} g_t$ .

**Shampoo** (Gupta et al., 2018)  $L_t = L_{t-1} + g_t g_t^\top$ ,  $R_t = R_{t-1} + g_t^\top g_t$ ,  $\theta_{t+1} = \theta_t - \eta L_t^{-1/4} g_t R_t^{-1/4}$ .

- Proposed by Bernstein & Newhouse (2024),  $\Delta\theta = -\eta(gg^\top)^{-1/4} g(g^\top g)^{-1/4} = -\eta \text{Polar}(g)$ .

**CASPR** (Duvvuri et al., 2024)  $L_t = L_{t-1} + g_t g_t^\top$ ,  $R_t = R_{t-1} + g_t^\top g_t$ ,  $\tilde{L}_t^{-1/4} = (L_t + \epsilon I_m)^{-1/4}$ ,  $\tilde{R}_t^{-1/4} = (R_t + \epsilon I_n)^{-1/4}$ ,  $U_t = \tilde{L}_t^{-1/4} g_t + g_t \tilde{R}_t^{-1/4}$ ,  $\theta_{t+1} = \theta_t - \eta(\tilde{L}_t^{-1/4} U_t + U_t \tilde{R}_t^{-1/4})$ .

**Orthogonal-SGDM** (Tuddenham et al., 2022)  $M_t = \gamma M_{t-1} + \eta \text{Polar}(g_t)$ ,  $\theta_{t+1} = \theta_t - M_t$ .

**Muon** (Jordan et al., 2024)  $M_t = \mu M_{t-1} + g_t$ ,  $O_t = \text{NewtonSchulz5}(M_t)$ ,  $\theta_{t+1} = \theta_t - \eta O_t$ . (In practice,  $\mu M_t + g_t$  is used in NewtonSchulz5 instead of  $M_t$ .)

- **Moonlight** (Liu et al., 2025a)  $\theta_{t+1} = \theta_t - \eta(0.2\sqrt{\max(m, n)} O_t + \lambda \theta_t)$

**AdaMuon** (Si et al., 2025) (Based on Muon)  $v_t = \mu v_{t-1} + (1 - \mu) O_t \odot O_t$ ,  $\hat{O}_t = \frac{O_t}{\sqrt{v_t + \epsilon}}$ ,  $\theta_{t+1} = \theta_t - \eta(\frac{0.2\sqrt{mn}}{\|\hat{O}_t\|_F} \hat{O}_t + \lambda \theta_t)$ .

**PolarGrad** (Lau et al., 2025)  $U_t H_t = \text{polar}(g_t)$ ,  $\theta_{t+1} = \theta_t - \eta \text{tr}(H_t) U_t$ .

- **PolarMuon**:  $m_t = \beta m_{t-1} + (1 - \beta) g_t$ ,  $U_t H_t = \text{polar}(m_t)$ ,  $\theta_{t+1} = \theta_t - \eta(\text{tr}(H_t) U_t + \lambda \theta_t)$ .
- **Polar-first**:  $U_t H_t = \text{polar}(g_t)$ ,  $m_t = \beta m_{t-1} + (1 - \beta) U_t$ ,  $\theta_{t+1} = \theta_t - \eta(\text{tr}(H_t) m_t + \lambda \theta_t)$ .

**AdaDiag** (Nguyen et al., 2025) If  $t \bmod T = 0$ ,  $P_t, Q_t^\top = \text{SVD}(g_t)$  else  $P_t, Q_t^\top = P_{t-1}, Q_{t-1}^\top$ .  $\tilde{g}_t = P_t^\top g_t$ ,  $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \tilde{g}_t$ ,  $v_t = \beta_2 v_{t-1} + (1 - \beta_2) \tilde{g}_t^2$ ,  $\theta_{t+1} = \theta_t - \eta_t(P_t \frac{m_t}{\sqrt{v_t + \epsilon}} + \lambda \theta_t)$

- **AdaDiag++**: similarly but  $\tilde{g}_t = P_t^\top g_t Q_t$ ,  $\theta_{t+1} = \theta_t - \eta_t(P_t \frac{m_t}{\sqrt{v_t + \epsilon}} Q_t^\top + \lambda \theta_t)$

**COSMOS** (Liu et al., 2025b)  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ ,  $u_t = \text{QR}(\beta_2 u_{t-1} s_{t-1} + (1 - \beta_2)g_t^\top g_t u_{t-1})$ ,  $s_t = u_t^\top (\beta_2 u_{t-1} s_{t-1} u_{t-1}^\top + (1 - \beta_2)g_t^\top g_t)u_t$ ,  $v_t = \beta_2 v_{t-1} + (1 - \beta_2)(g_t u_t) \odot (g_t u_t)$ ,  $a_t = (\frac{m_t u_t / (1 - \beta_1)}{\sqrt{(v_t + \epsilon) / (1 - \beta_2)}})u_t^\top$ ,  $b_t = \text{Norm}(\text{NewtonSchulz5}(\frac{m_t - m_t u_t u_t^\top}{\|m_t - m_t u_t u_t^\top\|_F}))$ ,  $\theta_{t+1} = \theta_t - \eta \text{Norm}(a_t + \gamma b_t \sqrt{m})$ , where  $\text{Norm}(X) = \frac{\sqrt{n}X}{\|X\|_F}$ .

**Dion** (Ahn et al., 2025b) (Centralized version)  $B_t = m_{t-1} + g_t$ . Then do Power Iteration for 1 iteration:  $P_t = \text{Orthogonalize}(B_t Q_{t-1})$  (orthogonalize  $P_t$  using Cholesky decomposition),  $R_t = B_t^\top P_t$ .  $m_t = B_t - (1 - \mu)P_t R_t^\top$ ,  $Q_t = \text{ColumnNormalize}(R_t)$  (Normalize each column of  $R_t$ ),  $\theta_t = \theta_{t-1} - \eta \sqrt{m/n} P_t Q_t^\top$ .

**Dion2** (Ahn et al., 2025a)  $m'_t = m_t + g_t$ . Select top rows/columns  $\mathcal{K}$  based on  $\ell_1$  norm or randomly select rows/columns  $\mathcal{K}$  of  $m_{t+1}$ ,  $O_t = \text{Ortho}(m_t[\mathcal{K}, :])$ ,  $\theta_{t+1}[\mathcal{K}, :] = \theta_t[\mathcal{K}, :] - \eta \sqrt{m/n} O_t$ ,  $m_{t+1}[\mathcal{K}, :] = \mu m'_t[\mathcal{K}, :]$ ,  $m_{t+1}[\neg \mathcal{K}, :] = m'_t[\neg \mathcal{K}, :]$ .

**MARS-M** (Liu et al., 2025c)  $c_t = g_t + \gamma_t \frac{\beta_1}{1 - \beta_1} (g_t - \nabla f(\theta_{t-1}, \xi_t))$ ,  $\tilde{c}_t = \text{Clip}(c_t, 1)$ ,  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)\tilde{c}_t$ ,  $\theta_{t+1} = \theta_t - \eta_t (0.2 \sqrt{\max(m, n)} \text{Polar}(m_t) + \lambda \theta_t)$ .

**NorMuon** (Li et al., 2025)  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ .  $O_t = \text{Polar}(m_t)$ ,  $v_t = \beta_2 v_{t-1} + (1 - \beta_2)\text{mean}_{\text{col}}(O_t \cdot O_t)$ ,  $V_t = \text{ExpandRows}(v_t)$ ,  $\hat{O}_t = O_t / (\sqrt{V_t} + \epsilon)$ ,  $\theta_{t+1} = \theta_t - \eta (\lambda \theta_t + 0.2 \sqrt{mn} \hat{O}_t / \|\hat{O}_t\|_F)$ .

**MuonAdam** (Crawshaw et al., 2025)  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ ,  $\theta_{t+1} = \theta_t - \eta_m \text{Polar}(m_t)$ . For 1D parameters, use Adam(W) with different learning rate.

**Conda** (Wang et al., 2025a) (Column-Normalized Adam) (Only for 2D parameters  $\theta \in \mathbb{R}^{m \times n}$ , assume  $m \leq n$ )  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ . If  $t \bmod T = 0$ ,  $U_t, \Sigma_t, V_t^\top = \text{SVD}(m_t)$ ,  $\bar{U}_t = U_t$ ; else  $\bar{U}_t = \bar{U}_{t-1}$ .  $m'_t = \bar{U}_t^\top m_t$ ,  $n_t = \beta_2 V_{t-1} + (1 - \beta_2)(\bar{U}_t^\top g_t)^2$ ,  $\theta_{t+1} = \theta_t + \eta \bar{U}_t \frac{m'_t}{\sqrt{n_t + \epsilon}} \frac{1 - \beta_2}{1 - \beta_1}$ .

**SSO** (Xie et al., 2026)  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $\hat{m}_t = m_t / \|m_t\|_F$ .  $(\sigma_t, u_t, v_t) = \text{PowerIteration}(\theta_t)$  (to get the top singular value and vectors),  $\Theta_t = u_t v_t^\top$ . Define  $R = \sqrt{d_{\text{out}}/d_{\text{in}}}$  as  $\mu P$  scaler.  $\lambda_t^* = \arg \min_{\lambda} \langle \Theta_t, \text{msign}(\hat{m}_t + \lambda \Theta_t) \rangle$  (using Bisection search with tolerance  $\epsilon$ ).  $\theta_{t+1} = \theta_t \cdot R / \sigma_t - \eta R \cdot \text{msign}(\hat{m}_t + \lambda_t^* \Theta_t)$ .

**M3** (Behrouz et al., 2025) (Multi-scale Momentum Muon) For frequency  $f$ , for low-frequency iteration  $k$ ,  $M_{kf}^{(2)} = M_{(k-1)f}^{(2)} + \beta_3 \sum_{t=(k-1)f}^{kf} g_t$ ,  $O_{kf}^{(2)} = \text{Polar}(M_{kf}^{(2)})$ . For  $t = kf + i$ ,  $M_t^{(1)} = M_{t-1}^{(1)} + \beta_1 g_t$ ,  $V_t = V_{t-1} + \beta_2 g_t^2$ ,  $O_t^{(1)} = \text{Polar}(M_t^{(1)})$ ,  $\theta_{t+1} = \theta_t - \eta \frac{O_t^{(1)} + \alpha O_t^{(2)}}{\sqrt{V_t + \epsilon}}$ .

**AdaGO** (Wang et al., 2025c)  $M_t = \mu M_{t-1} + (1 - \mu)g_t$ ,  $v = v_{t-1} + \min\{\|g_t\|^2, \gamma^2\}$ ,  $\theta_{t+1} = \theta_t - \max\{\epsilon, \eta \frac{\min\{\|g_t\|, \gamma\}}{\sqrt{v_t}}\} \text{Polar}(M_t)$ .

**MoFaSGD** (Mahdavinia & Mahdavi, 2025) Initialize  $U_0, \Sigma_0, V_0^\top = \text{SVD}(g_0)$ ,  $(U'_t, R_{U_t}) = \text{QR}([U_t, g_t V_t])$ ,  $(V'_t, R_{V_t}) = \text{QR}([V_t, G_t^\top U_t])$ ,  $S_t = R_{U_t} \begin{bmatrix} \beta \Sigma_t - U_t^\top g_t V_t & I_r \\ I_r & 0_r \end{bmatrix} R_{V_t}^\top$ ,  $U_t'' \Sigma_t'' (V_t'')^\top = \text{SVD}(S_t)$ ,  $U_{t+1} = U'_t U_t''$ ,  $V_{t+1} = V'_t V_t''$ ,  $\Sigma_{t+1} = \Sigma_t''$ ,  $\theta_{t+1} = \theta_t - \eta U_{t+1} V_{t+1}^\top$ .

**GUM** (Pan et al., 2025)  $U_t, \Sigma_t, V_t = \text{SVD}(g_t)$ ,  $P_t = U_t[:, :r]$  for low rank  $r$ . For each  $\theta_{tK+k}$ , with probability  $q$ , do  $K$  iterations with  $m_{tK+k} = \beta m_{tK+k-1} + \frac{g_t - P_t P_t^\top g_t}{q}$ ,  $\theta_{tK+k+1} = \theta_{tK+k} + \eta \text{Polar}(m_{tK+k})$ ; otherwise do  $K$  iterations with  $m_{tK+k} = \beta m_{tK+k-1} + \frac{P_t^\top g_t}{1-q}$ ,  $\theta_{tK+k+1} = \theta_{tK+k} + \eta P_t \text{Polar}(m_{tK+k})$ .

**Muon-NSR** (Li et al., 2026)  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $v_t = \beta v_{t-1} + \beta(1 - \beta)(m_{t-1} - g_t)^2$ ,  $\hat{m}_t = \frac{m_t}{1 - \beta^t}$ ,  $\hat{v}_t = \frac{v_t}{1 - \beta^t}$ ,  $\tilde{m}_t = g_t + \frac{\beta}{1 - \beta} \hat{m}_t$ ,  $\bar{m}_t = \frac{\tilde{m}_t}{\sqrt{\tilde{m}_t^2 + \gamma \hat{v}_t + \epsilon}}$ ,  $\theta_{t+1} = \theta_t(1 - \eta \lambda) - \eta s_{\text{scale}} \text{Polar}(\bar{m}_t)$ .

**Muon-VS** (Li et al., 2026) Same as Muon-NSR, but  $\bar{m}_t = \frac{\tilde{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$ .

**Muon+** (Zhang et al., 2026b)  $m_t = \mu m_{t-1} + (1 - \mu)g_t$ ,  $O_t = \text{Norm}(\text{Polar}(m_t))$ ,  $\theta_{t+1} = \theta_t - \eta \sqrt{m/n} O_t$ , where Norm is column-wise or row-wise norm, or column-then-row/row-then-column norm.

**NAMO** (Zhang et al., 2026a)  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ ,  $v_t = \beta_2 v_{t-1} + (1 - \beta_2)\|g_t\|_F^2$ ,  $O_t = \text{Ortho}(m_t)$ ,  $\theta_{t+1} = \theta_t - \eta \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t} \frac{\|m_t\|_F}{\sqrt{v_t+\epsilon}} O_t$ .

**PRISM** (Yang, 2026)  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $d_t = g_t - m_t$ ,  $O_t = \text{Polar}(\text{Concat}[m_t; \gamma d_t])$ ,  $\theta_{t+1} = \theta_t - \eta O_t[:, m, :]$ .

**Aurora** (Research, 2026) (Only for 2D parameters  $\theta \in \mathbb{R}^{m \times n}$ )  $X_0 = \theta_t / \|\theta_t\|_F$ ,  $D_0 = I_m$ . For  $k = 0, \dots, k-1$ ,  $r_k = (\|(X_k)_{1,:}\|, \dots, \|(X_k)_{m,:}\|)$ ,  $D_k = D_{k-1}^\beta \cdot \text{diag}(r_k)^{1-\beta}$ ,  $X_{k+1} = \text{Polar}(\sqrt{n/m} D_k^{-1} X_k)$ , where  $\beta$  is 0.5 by default.

**HTMuon** (Pang et al., 2026)  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $U_t, \Sigma_t, V_t^\top = \text{SVD}(m_t)$ ,  $O_t = U_t \Sigma_t^p V_t^\top$ ,  $\theta_{t+1} = \theta_t - \eta(\lambda \theta_t + \sqrt{\max(1, m/n)} O_t)$ .

- **HTMuon-NS** Compute  $O_t$  via Newton-Schulz approximation:  $O'_t = \text{NewtonSchulz5}(m_t)$ ,  $O''_t = \text{NS-root}(m_t^\top m_t)$ ,  $O_t = O'_t O''_t$ ;
- Here NS-root( $X$ ) is: for  $s = 1, \dots, \lceil \log_2 \frac{2}{p} \rceil$ ,  $\alpha = \|X_s\|_F$ ,  $Y_{s,0} = X_s$ ,  $Z_{s,0} = I$ , iterative for  $T$  times,  $Q_{s,i} = 3I - Z_{s,i-1}$ ,  $Y_{s,i} = Y_{s,i-1} Q_{s,i}/2$ ,  $Z_{s,i} = Q_{s,i} Z_{s,i-1}/2$ , and update  $X_{s+1} = \sqrt{\alpha \frac{Y_{s,T} + Y_{s,T}^\top}{2}} + \epsilon I$ . Finally return  $X_{\lceil \log_2 \frac{2}{p} \rceil + 1}$ .

**Newton-Muon** (Du & Su, 2026) (Only for 2D parameters  $\theta \in \mathbb{R}^{m \times n}$ , based on Muon) For each  $k$  steps,  $K_t = \beta K_{t-1} + (1 - \beta)Z_t Z_t^\top / N$  for input activations  $Z_t \in \mathbb{R}^{n \times N}$ ,  $\gamma_t = \gamma \text{tr}(K_t)/n$ ,  $K_t^{-1} = (K_t + \gamma_t I_n)^{-1}$ , computed by Cholesky inverse or a polynomial iteration with a custom symmetric-output matmul; otherwise  $K_t = K_{t-1}$ . Apply Muon optimizer with modified gradient  $g_t K_t^{-1}$ .

## 5.5 Miscellaneous

**BFGS** (Broyden–Fletcher–Goldfarb–Shanno) (Fletcher, 1987)  $d_t = -H_t g_t$ ,  $\alpha_t = \arg \min_\alpha f(\theta - \alpha d_t)$ ,  $\theta_{t+1} = \theta_t + \alpha_t d_t$ ,  $H_{t+1} = (I - \frac{s_t y_t^\top}{y_t^\top s_t})^\top H_t (I - \frac{y_t s_t^\top}{y_t^\top s_t}) + \frac{s_t s_t^\top}{y_t^\top s_t}$ , where  $s_t = \theta_{t+1} - \theta_t$ ,  $y_t = g_{t+1} - g_t$ .

**L-BFGS** (Liu & Nocedal, 1989) Based on BFGS, but choose  $\alpha_t$  satisfying Wolfe conditions (try  $\alpha_t = 1$  first):  $f(\theta_t + \alpha_t d_t) \leq f(\theta_t) + \beta' \alpha_t g_t^\top d_t$ . Moreover, for  $\hat{m} = \min\{t, m-1\}$ ,

$$H_{t+1} = (\prod_{i=t, inv}^{t-\hat{m}} V_i^\top) H_0 (\prod_{i=t-\hat{m}}^t V_i) + \sum_{j=t-\hat{m}}^t \rho_j (\prod_{i=t, inv}^{j+1} V_i^\top) s_j s_j^\top (\prod_{i=j+1}^t V_i),$$

where  $\prod_{i=t, inv}^{t'}$  denotes the product of matrices with indices from  $t$  to  $t' < t$ , and  $\rho_t = 1/(y_t^\top s_t)$ .

**SOAP** (Vyas et al., 2024) (Only for 2D parameters)  $g'_t = Q_L^\top g Q_R$ ,  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ ,  $m'_t = Q_L^\top m_t Q_R$ ,  $v_t = \beta_2 v_{t-1} + (1 - \beta_2)(g'_t)^2$ ,  $\theta_{t+1} = \theta_t - \eta Q_L \frac{\hat{m}'_t}{\sqrt{v_t+\epsilon}} Q_R^\top$ , where  $\hat{m}'_t = \frac{m'_t}{1-\beta_1^t}$ ,  $\hat{v}_t = \frac{v_t}{1-\beta_2^t}$ .  $L_t = \beta_2 L_{t-1} + (1 - \beta_2)g g^\top$ ,  $R_t = \beta_2 R_{t-1} + (1 - \beta_2)g^\top g$ . For every  $k$  steps, obtain  $Q_L = \text{QR-EigenVectors}(L Q_L)$ ,  $Q_R = \text{QR-EigenVectors}(R Q_R)$ , where QR-EigenVectors returns the eigenvector matrix of the QR decomposition for the input matrix.

**Hessian-free** (Martens et al., 2010) Define the function  $B_n(d) = \mathbf{H}(\theta_t)d + \lambda d$ , where  $\mathbf{H}(\theta_t)d = \lim_{\epsilon \rightarrow 0} \frac{\nabla f(\theta_t + \epsilon d) - g_t}{\epsilon}$ .  $p_t = \text{CG-Minimize}(B_t, -g_t)$ , where CG-Minimize is the linear conjugate gradient algorithm,  $\theta_{t+1} = \theta_t + p_t$ .  $\lambda$  can be adjusted by Levenberg-Marquardt style heuristic: for  $\rho_t = \frac{f(\theta_t + p_t) - f(\theta_t)}{q_{\theta_t}(p) - q_{\theta_t}(0)}$  with  $q_{\theta_t}(\cdot)$  the minimization objective of CG, if  $\rho_t < \frac{1}{4}$ ,  $\lambda \rightarrow \frac{3}{2}\lambda$ , else if  $\rho_t > \frac{3}{4}$ ,  $\lambda \rightarrow \frac{2}{3}\lambda$ , otherwise keep  $\lambda$  unchanged.

**Formage** (Bernstein et al., 2020) (For each layer)  $\theta_{t+1} = \frac{1}{\sqrt{1+\eta^2}}(\theta_t - \eta \frac{\|\theta_t\|_F}{\|g_t\|_F} g_t)$ .

**SWAN** (Ma et al., 2024) (Only for 2D parameters  $\theta \in \mathbb{R}^{m \times n}$ )  $g'_t = \frac{g_t}{\sqrt{\frac{1}{n} \sum_{j=1}^n (g_{t, [\cdot, j]})^2}} \mathbf{1}_n^\top$ ,  $Y_{t,0} = g'_t g'_t^\top$ ,  $Z_{t,0} = I$  for  $K$  iterations,

- if use diagonal method,  $Y_{t,k} = \beta Y_{t,k-1} \text{diag}(3I - Z_{t,k} \text{diag}(Y_{t,k}))$ ,  $Z_{t,k} = \beta(3I - \text{diag}(Z_{t,k} Y_{t,k})_{t,k} (Z_{t,k}))$ ;
- otherwise  $Y_{t,k} = \beta Y_{t,k}(3I - Z_{t,k} Y_{t,k})$ ,  $Z_{t,k} = \beta(3I - Z_{t,k} Y_{t,k}) Z_{t,k}$ .

Then  $\Delta_t = Z_{t,k} g_t$ . (optional:  $\Delta_t = \frac{\sqrt{mn} \Delta_t}{\|\Delta_t\|} \theta_{t+1} = \theta_t - \eta \Delta_t$ ).

**ROOT** (He et al., 2025b) (Only for 2D parameters  $\theta \in \mathbb{R}^{m \times n}$ )  $m_t = \mu m_{t-1} + g_t$ ,  $O_t = \text{sign}(m_t) \cdot \max(|m_t| - \epsilon, 0)$ ,  $B_t = m_t - O_t$ ,  $\tilde{B}_t = \text{AdaNewton}(B_t)$ ,  $\theta_{t+1} = \theta_t - \eta \tilde{B}_t$ . Here AdaNewton is similar to Newton-Schulze iteration but  $(a, b, c) \in \mathbb{R}^{m \times n}$  and they are trainable for each parameter.

**Mano** (Gu & Xie, 2026) (Only for 2D parameters  $\theta \in \mathbb{R}^{m \times n}$ )  $m_t = \mu m_{t-1} + g_t$ ,  $k = t \bmod 2$ ,  $v_t = m_t - \hat{\theta}_t \odot \langle m_t, \hat{\theta}_t \rangle_k$  (here the inner-product is done on  $k$ -th dimension),  $\theta_{t+1} = \theta_t - \eta(0.2\sqrt{n}\hat{v}_t + \lambda\theta_t)$ , where  $\hat{\theta}_t, \hat{v}_t$  is column-wise or row-wise norm of  $\theta_t, v_t$  when  $k$  is 0 or 1.

## 6 Variance Reduction

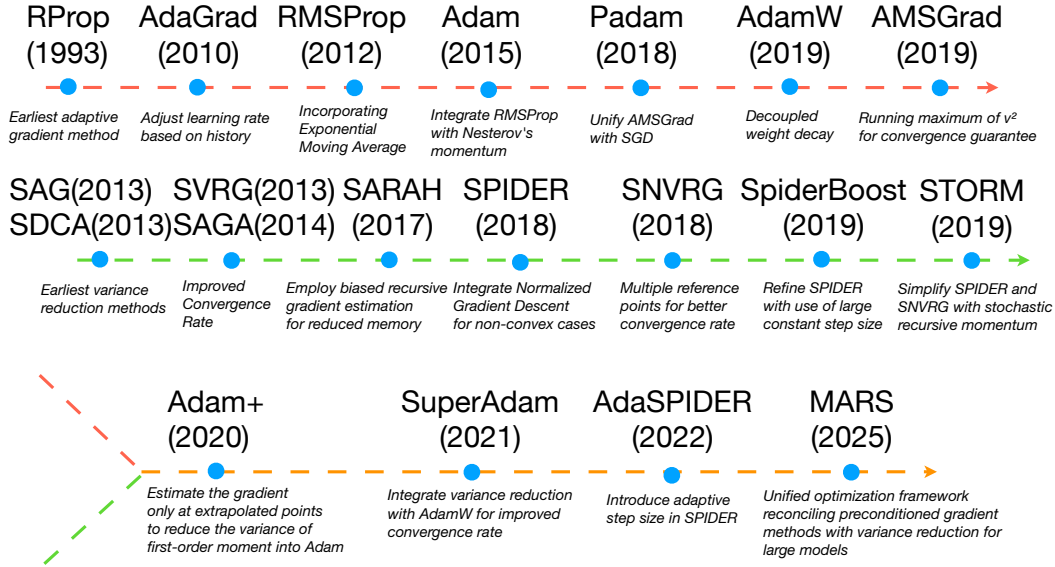


Figure 3: A brief history of variance reduction optimization methods. Some optimizers are omitted.

Since adaptive gradient methods may have risks of high stochastic gradient variance, some researchers consider variance reduction techniques to address this challenge. The core idea, first proposed by Johnson & Zhang (2013), is as follows:

$$\mathbf{m}_t = \nabla f(\theta_t, \xi_t) - \nabla f(\tilde{\theta}_t, \xi_t) + \nabla F(\tilde{\mathbf{x}}),$$

where  $\tilde{\mathbf{x}}$  is some reference point (anchoring point) that is periodically updated. By differentiating the gradient for different points on the same data point, the variance in optimization can be reduced significantly.

**SAG** (Roux et al., 2012)  $\theta_{t+1} = \theta_t - \frac{\eta}{n} \sum_{i=1}^n y_{i,t}$ , where at each  $t$ , one  $\xi_{i_t}$  is chosen and  $y_{i,t} = \nabla f(\theta_t, \xi_{i,t})$  and for other samples, keep  $y_{\cdot,t}$  unchanged.

**SDCA** (Shalev-Shwartz & Zhang, 2013) (For machine learning task  $L(\theta) = \frac{1}{n} \sum_{i=1}^n \phi_i(\theta^\top \xi_i) + \frac{\lambda}{2} \|\theta\|^2$  with scalar convex function  $\phi_i$ ), for  $\xi_t = \xi_i$ ,  $\Delta\alpha_t = \arg \max_{\Delta\alpha} -\phi_i^*(-(\alpha_{t-1} + \Delta\alpha), \xi_t) -$

$\frac{\lambda n}{2} \|\theta_{t-1} + \frac{1}{\lambda n} \xi_t \Delta \alpha\|^2$ ,  $\alpha_t = \alpha_{t-1} + \Delta \alpha_t e_i$ ,  $\theta_t = \theta_{t-1} + (\lambda n)^{-1} \xi_t \Delta \alpha_t$ . Output  $\frac{1}{T} \sum_{i=1}^T \theta_{t-1}$  or randomly chosen from  $\{\theta_t\}_{t=1}^T$ .

**SAGA** (Defazio et al., 2014) Keep a  $n \times d$  matrix  $\phi = \{\nabla f(\theta_0, \xi_i)\}_{i=1}^n$  storing the parameters, for  $\xi_t = \xi_j$ ,  $\theta_{t+1} = \arg \min_{\theta} \{h(\theta) + \frac{1}{2\gamma} \|\theta - (\theta_t - \gamma[\nabla f(\theta_t, \xi_t) - \phi_j + \frac{1}{n} \sum_{i=1}^n \phi_i])\|^2\}$ , where  $h(\theta)$  is the regularization function, then update  $\phi_j = \theta_t$  and keep other  $\phi_i$  unchanged.

**SVRG** (Johnson & Zhang, 2013) For each epoch  $s$ , calculate the full gradient  $g_s$  for  $\theta_s$ . For each iteration  $t$ ,  $\theta_{s,t+1} = \theta_{s,t} - \eta(\nabla f(\theta_{s,t}, \xi_{s,t}) - \nabla f(\theta_s, \xi_{s,t}) + g_s)$ , set  $\theta_s = \theta_{s,T}$  or randomly chosen from  $\{\theta_{s,t}\}_{t=1}^T$ .

- **$\alpha$ -SVRG** (Yin et al., 2025)  $\theta_{s,t+1} = \theta_{s,t} - \eta(\alpha_t(\nabla f(\theta_{s,t}, \xi_{s,t}) - \nabla f(\theta_s, \xi_{s,t})) + g_s)$ , where  $\alpha_t = \frac{\text{Cov}(\nabla f(\theta_{s,t}, \xi_{s,t}), \nabla f(\theta_{s,t}, \xi_{s,t}))}{\text{Var}(\nabla f(\theta_{s,t}, \xi_{s,t}))}$ .

**SARAH** (Nguyen et al., 2017) For each epoch  $s$ , calculate the full gradient  $v_{s,0} = g_s$  for  $\theta_s$ ,  $\theta_{s,1} = \theta_{s,0} - \eta v_{s,0}$ . For each iteration  $t$ ,  $v_{s,t} = \nabla f(\theta_{s,t}, \xi_{s,t}) - \nabla f(\theta_s, \xi_{s,t}) + v_{s,t-1}$ ,  $\theta_{s,t+1} = \theta_{s,t} - \eta v_{s,t}$ , set  $\theta_s$  randomly chosen from  $\{\theta_{s,t}\}_{t=1}^T$ .

**Hybrid-SGD** (Tran-Dinh et al., 2019) (Hybrid SGD with SARAH)  $m_t = \beta(m_{t-1} + g_t - \nabla f(\theta_{t-1}, \xi_t)) + (1 - \beta)g_t$ ,  $\theta_{t+1} = \theta_t - \eta m_t$

**SPIDER-SFO** (Fang et al., 2018)  $v_t = g_t$  for every  $q$  steps with batch size  $S_1$ , otherwise  $v_t = g_t - \nabla f(\theta_{t-1}, \xi_t) + v_{t-1}$ . Update parameters with Option I:  $\theta_{t+1} = \theta_t - \eta v_t / \|v_t\|$  until  $\|v_t\|$  less than some threshold, or Option II:  $\theta_{t+1} = \theta_t - \eta_t v_t$  with  $\eta_t = \min(\eta / \|v_t\|, \eta / (2\epsilon))$ . Output  $\theta_T$  or randomly choose from  $\{\theta_t\}_{t=1}^T$ .

**SpiderBoost** (Wang et al., 2019)  $v_t = g_t - \nabla f(\theta_{t-1}, \xi_t) + v_{t-1}$ ,  $\theta_{t+1} = \theta_t - \eta v_t / \|v_t\|$

**AdaSpider** (Kavis et al., 2022) If  $t \bmod n = 0$ ,  $v_t = g_t$ , else  $v_t = g_t - \nabla f(\theta_{t-1}, \xi_t) + v_{t-1}$ .  $\gamma_t = 1 / \left( n^{1/4} \beta_0 \sqrt{n^{1/2} G_0^2 + \sum_{s=0}^t \|v_s\|^2} \right)$ ,  $\theta_{t+1} = \theta_t - \gamma_t v_t$ .

**SNVRG** (Zhou et al., 2020b) For loop parameters  $\{T_l\}$ , set  $T = \prod_{l=1}^K T_l$  or  $T \sim \text{Geom}(1/(1 + \prod_{l=1}^K T_l))$  as the total number of steps. For each step  $t$ ,  $r_t = \min\{j : 0 = (t \bmod \prod_{l=j+1}^K T_l), 0 \leq j \leq K\}$ . For  $0 \leq l \leq r_t - 1$ ,  $\theta_t^l = \theta_{t-1}^l$ ; otherwise  $\theta_t^l = \theta_t$ . For  $0 \leq l \leq r_t - 1$ ,  $g_t^l = g_{t-1}^l$ ; for  $r_t + 1 \leq l \leq K$ ,  $g_t^l = 0$ . Then uniformly generate index set  $I_t$  with size  $B_{r_t}$ , if  $r_t > 0$ ,  $g_t^{r_t} = \frac{1}{B_{r_t}} \sum_{i \in I_t} [\nabla f(\theta_t^{r_t}, \xi_i) - \nabla f(\theta_t^{r_t-1}, \xi_i)]$ , otherwise  $g_t^0 = \frac{1}{B_0} \sum_{i \in I_t} \nabla f(\theta_t^0, \xi_i)$ . And  $\theta_{t+1} = \theta_t - \eta \sum_{l=0}^K g_t^l$ . The output parameter is randomly chosen from  $\{\theta_t\}_{t=1}^T$ .

**STORM<sup>+</sup>** (Levy et al., 2021) (Based on STORM)  $a_{t+1} = \frac{1}{(1 + \sum_{i=1}^t \|g_i\|^2)^{2/3}}$ ,  $\eta_t = \frac{1}{(\sum_{i=1}^t \|d_i\|^2 / a_{i+1})^{1/3}}$

**Super-Adam** (Huang et al., 2021) (Based on Adam)  $c_t = \alpha_t g_t + (1 - \alpha_t)[c_{t-1} + \tau(g_t - \nabla f(\theta_{t-1}, \xi_t))]$ ,  $\tau \in \{0, 1\}$ ,  $\tilde{\theta}_t = \arg \min_{\theta} \{\eta \langle c_t, \theta \rangle + \frac{1}{2} \|\theta - \theta_t\|_{\mathbf{H}_t}^2\}$ ,  $\theta_{t+1} = (1 - \mu_t)\theta_t + \mu_t \tilde{\theta}_t$ , where  $\mathbf{H}_t$  is defined by one of the following cases:

- Case 1:  $\mathbf{H}_t = \text{diag}(\sqrt{v_t} + \lambda)$
- Case 2:  $v_t = \beta v_{t-1} + (1 - \beta)\|g_t\|$ ,  $\mathbf{H}_t = (v_t + \lambda)I_d$
- Case 3 (Barzilai-Borwein technique):  $b_t = \frac{\|\langle g_t - \nabla f(\theta_{t-1}, \xi_t), \theta_t - \theta_{t-1} \rangle\|}{\|\theta_t - \theta_{t-1}\|}$ ,  $\mathbf{H}_t = (b_t + \lambda)I_d$
- Case 4-1:  $v_t = \beta_2 v_{t-1} + (1 - \beta_2)(g_t - m_t)^2$ ,  $\mathbf{H}_t = \text{diag}(\sqrt{v_t} + \lambda)$
- Case 4-2:  $v_t = \beta_2 v_{t-1} + (1 - \beta_2)\|g_t - m_t\|$ ,  $\mathbf{H}_t = (v_t + \lambda)I_d$

**ROOT-SGD** (Li et al., 2022)  $v_t = g_t + \frac{t-1}{t}(v_{t-1} - \nabla f(\theta_{t-1}, \xi_t))$ ,  $\theta_{t+1} = \theta_t - \eta v_t$

**AdaSVRPS/AdaSVRLS** (Jiang & Stich, 2024)  $F_{\xi_t}(\theta) = f(\theta, \xi_t) + \mathbf{w}^\top (\nabla f(\mathbf{w}_t) - \nabla f(\mathbf{w}_t, \xi_t)) + \frac{\mu_F}{2} \|\theta - \theta_t\|^2$ ,  $\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} F_{\xi_t}(\theta_t)$ . With probability  $p_{t+1}$ ,  $\mathbf{w}_{t+1} = \theta_t$ , otherwise  $\mathbf{w}_{t+1} = \mathbf{w}_t$ . Output  $\frac{1}{T} \sum_{t=0}^T \theta_t$ . Here



- $\eta_t = \min\left\{\frac{F_{\xi_t}(\theta_t) - F_{\xi_t}^*}{c_p \|\nabla F_{\xi_t}(\theta_t)\|^2 \sqrt{\sum_{s=0}^t F_{\xi_s}(\theta_s) - F_{\xi_s}^*}}, \eta_{t-1}\right\}$  for AdaSVRPS
- $\eta_t = \min\left\{\gamma_t \frac{1}{c_t \sqrt{\sum_{s=0}^t \gamma_s \|\nabla F_{\xi_s}(\theta_s)\|^2}}, \eta_{t-1}\right\}$  for AdaSVRLS, where  $\gamma_t$  can be obtained by Armijo Backtracking line-search (Armijo, 1966; Nocedal & Wright, 2006): do  $\gamma = \beta\gamma$  until  $f(\theta_t - \gamma \nabla f(\theta_t, \xi_t), \xi_t) \leq f(\theta_t, \xi_t) - \rho\gamma \|\nabla f(\theta_t, \xi_t)\|^2$
- For AdaSPS and AdaSLS, just set  $F_{\xi_t}(\theta) = f(\theta, \xi_t)$  and set  $F_{\xi_t}^*$  as a predefined lower bound.

**VRAdam** (Li et al., 2023) (Based on Adam)  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)(g_t + \frac{\beta_1}{1 - \beta_1}(g_t - \nabla f(\theta_{t-1}, \xi_t)))$ ,  $\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{\hat{v}_t + \epsilon}}$

**MARS** (Yuan et al., 2025) (Based on AdamW/Lion/Shampoo)  $c_t = g_t + \gamma_t \frac{\beta_1}{1 - \beta_1}(g_t - \nabla f(\theta_{t-1}, \xi_t))$ ,  $\tilde{c}_t = \text{Clip}(c_t, 1)$ ,  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)\tilde{c}_t$ ,  $\theta_{t+1} = \arg \min_{\theta} \{\eta \langle m_t, \theta \rangle + \frac{1}{2} \|\theta - \theta_t\|_{\mathbf{H}_t}^2\}$ , or  $\theta_{t+1} = \arg \min_{\theta} \{\eta \langle m_t, \theta \rangle + \frac{1}{2} \|\theta - (1 - \eta\lambda)\theta_t\|_{\mathbf{H}_t}^2\}$  with weight decay.

- MARS-AdamW:  $v_t = \beta_2 v_{t-1} + (1 - \beta_2)\tilde{c}_t^2$ ,  $\mathbf{H}_t := \sqrt{\text{diag}(v_t)} \cdot \frac{1 - \beta_1^t}{\sqrt{1 - \beta_2^t}}$ , (equivalently,  $\theta_{t+1} = \theta_t - \eta(\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda\theta_t)$ )
- MARS-Lion:  $\mathbf{H}_t = \sqrt{\text{diag}(m_t^2)}$ , (equivalently,  $\theta_{t+1} = \theta_t - \eta(\text{Sign}(m_t) + \lambda\theta_t)$ )
- MARS-Shampoo:  $\mathbf{H}_t = (\sum_{\tau=1}^t g_{\tau} g_{\tau}^{\top})^{1/4} \otimes (\sum_{\tau=1}^t g_{\tau}^{\top} g_{\tau})^{1/4}$ , (equivalently,  $U_t, \Sigma_t, V_t = \text{SVD}(m_t)$ ,  $\theta_{t+1} = \theta_t - \eta(U_t V_t^{\top} + \lambda\theta_t)$ ), and use Newton-Schulz iteration methods to approximate SVD decomposition (See Section 5.4 for detail). See MARS-M (Liu et al., 2025c) in Section 5.4 for a practical variant.
- MARS-Approximate:  $c_t = g_t + \gamma_t \frac{\beta_1}{1 - \beta_1}(g_t - g_{t-1})$ .
- MVR1 (Chang et al., 2025)  $\theta_{t+1} = \theta_t - \eta_t O_t$ , where  $O_t \in \arg \min_O \|O - m_t\|_F$  such that  $O^{\top} O = I_n$ . And MVR2 is the approximate version of MVR1.

**LiMuon** (Huang et al., 2025a)  $m_t = g_t + (1 - \beta)(m_{t-1} - \nabla f(\theta_{t-1}, \xi_t))$ ,  $U_t, \Sigma_t, V_t = \text{SVD}(m_t)$ ,  $\theta_{t+1} = \theta_t - \eta U_t V_t^{\top}$ .

## 7 Other Topics

### 7.1 Scheduler-Free/Parameter-Free Methods

**Schedule-Free AdamW** (Defazio et al., 2024)

- (Form 1)  $y_t = (1 - \beta_1)z_t + \beta_1\theta_t$ ,  $g_t = \nabla_{\theta} J(y_t)$ ,  $\eta_t = \eta \sqrt{1 - \beta_2^t} \min(1, t/T_{\text{warmup}})$ ,  $z_{t+1} = z_t - \eta_t(\frac{g_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda y_t)$ ,  $\theta_{t+1} = (1 - c_{t+1})\theta_t + c_{t+1}z_{t+1}$ , where  $c_{t+1} = \frac{\eta_t^2}{\sum_{i=1}^t \eta_i^2}$
- (Form 2)  $\eta_t = \eta \sqrt{1 - \beta_2^t} \min(1, t/T_{\text{warmup}})$ ,  $g_t = \nabla_{\theta} J(y_t)$ ,  $\Delta_t = \eta_t(\frac{g_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda y_t)$ ,  $y_{t+1} = y_t + \frac{\beta_1 c_{t+1}}{1 - \beta_1}(y_t - \theta_t) - [\beta_1 c_{t+1} + (1 - \beta_1)]\Delta_t$ ,  $\theta_{t+1} = \theta_t + \frac{c_{t+1}}{1 - \beta_1}(y_t - \theta_t) - c_{t+1}\Delta_t$ , where  $c_{t+1} = \frac{\eta_t^2}{\sum_{i=1}^t \eta_i^2}$

**D-Adaptation** (Defazio & Mishchenko, 2023)

- Dual Averaging:  $m_t = m_{t-1} + d_{t-1}g_t$ ,  $\eta_t = \frac{1}{\sqrt{\sum_{i=0}^k \|g_i\|^2}}$ . For  $\hat{d}_t$ , option I:  $\hat{d}_t = \frac{\eta_t \|m_t\|^2 - \sum_{i=0}^{t-1} \eta_i d_i^2 \|g_{i+1}\|^2}{2\|m_t\|}$  or option II:  $\hat{d}_t = \frac{\sum_{i=0}^{t-1} d_i \eta_i \langle g_{i+1}, m_i \rangle}{\|m_t\|}$ ,  $d_t = \max(d_{t-1}, \hat{d}_t)$ ,  $\theta_{t+1} = \theta_t - \eta_t m_t$ . Output  $\frac{1}{\sum_{t=0}^{T-1} d_t} \sum_{t=1}^T d_{t-1} \theta_t$
- Gradient Descent:  $\eta_t = \frac{d_{t-1}}{\sqrt{G^2 + \sum_{i=0}^t \|g_i\|^2}}$ ,  $m_t = m_{t-1} + \eta_t g_t$ ,  $\hat{d}_t = \frac{\|m_t\|^2 - \sum_{i=0}^t \eta_i^2 \|g_{i+1}\|^2}{2\|m_t\|}$ ,  $d_t = \max(d_{t-1}, \hat{d}_t)$ ,  $\theta_{t+1} = \theta_t - \eta_t m_t$ . Output  $\frac{1}{\sum_{t=0}^T \eta_t} \sum_{t=0}^T \eta_t \theta_t$



- **AdaGrad**:  $m_t = m_{t-1} + d_{t-1}g_t$ ,  $a_t^2 = a_{t-1}^2 + g_t^2$ ,  $A_t = \text{diag}(a_t)$ ,  $\hat{d}_t = \frac{\|m_t\|_{A_t^{-1}}^2 - \sum_{i=0}^{t-1} d_i^2 \|g_{i+1}\|_{A_i^{-1}}^2}{2\|m_t\|_1}$ ,  $d_t = \max(d_{t-1}, \hat{d}_t)$ ,  $\theta_{t+1} = \theta_t - A_t^{-1}m_t$ . Output  $\frac{1}{\sum_{t=0}^{T-1} d_t} \sum_{t=1}^T d_{t-1}\theta_t$
- **SGD**:  $\eta_k = \eta d_{k-1}/G$ ,  $m_t = m_{t-1} + \eta_t g_t$ ,  $z_t = z_{t-1} - \eta_t g_t$ ,  $\theta_{t+1} = \beta\theta_t + (1-\beta)z_t$ ,  $\hat{d}_t = \frac{2\sum_{i=0}^{t-1} \eta_{i+1} \langle g_{i+1}, m_i \rangle}{\|m_t\|}$ ,  $d_t = \max(d_{t-1}, \hat{d}_t)$ .
- **Adam version (Based on Adam)**  $m_t = \beta_1 m_{t-1} + (1-\beta_1)\eta d_{t-1}g_t$ ,  $\theta_{t+1} = \theta_t - \frac{m_t}{\sqrt{v_t + \epsilon}}$ ,  $s_t = \sqrt{\beta_2} s_{t-1} + (1-\sqrt{\beta_2})\eta d_{t-1}g_t$ ,  $r_t = \sqrt{\beta_2} r_{t-1} + (1-\sqrt{\beta_2})\eta d_{t-1} \langle g_t, s_{t-1} \rangle_{(\text{diag}(\sqrt{v_t + \epsilon}))^{-1}}$ ,  $\hat{d}_t = \frac{r_t}{(1-\sqrt{\beta_2})\|s_t\|_1}$ ,  $d_t = \max(d_{t-1}, \hat{d}_t)$

**Adam++** (Tao et al., 2024)  $\eta_t = \max(\eta_{t-1}, \|\theta_t - \theta_0\|/\sqrt{d})$ ,  $\beta_{1,t} = \beta_1 \lambda^{t-1}$ ,  $m_t = \beta_{1,t} m_{t-1} + (1-\beta_{1,t})g_t$ ,  $\theta_{t+1} = \theta_t - \eta_t \cdot \mathbf{H}_t^{-1} m_t$ , where  $\mathbf{H}_t = \epsilon + \text{diag}(s_t)$ , and  $s_t$  is calculated by either of the following ways:

- Case I:  $s_t = \sqrt{\sum_{i=0}^t g_i^2}$
- Case II:  $v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$ ,  $s_t = \sqrt{(t+1) \max_{t' \leq t} (v_{t'})}$

**AdaGrad++** (Tao et al., 2024) Similar to Adam++, but only choose Case I of  $s_t$ , and  $\lambda = 0$ .

**DoG** (Ivgi et al., 2023)  $\theta_{t+1} = \theta_t - \frac{\max_{i \leq t} \|x_i - x_0\|}{\sqrt{\sum_{i \leq t} \|g_i\|^2}} g_t$ .

**DoWG** (Khaled et al., 2023)  $r_t = \max(\|\theta_t - \theta_0\|, r_{t-1})$ ,  $v_t = v_{t-1} + r_t^2 \|g_t\|^2$ ,  $\theta_{t+1} = \theta_t - \frac{r_t^2}{\sqrt{v_t}} g_t$ .

**Mechanic** (Cutkosky et al., 2023) For the update of a basic optimizer  $\Delta_t$ , (optional) set  $d_t = \frac{\theta_t - \theta_0}{\bar{s}_t + \epsilon}$ ,  $d_{t+1} = d_t + \Delta_t$ ,  $h_t = \langle d_t, g_t + \frac{\lambda \bar{s}_t \|g_t\| \|\theta_t\|}{\|\theta_t\|} \rangle$ ,  $m_t = \max(\beta m_{t-1}, h_t)$ ,  $v_t = \beta^2 v_{t-1} + h_t^2$ ,  $r_t = \max(0, \beta r_{t-1} - s_{t-1} h_t)$ ,  $w_t = \frac{s_{\text{init}} m_t}{n} + r_t$ ,  $s_{t+1} = \frac{w_t}{\sqrt{v_t + \epsilon}}$ ,  $\bar{s}_{t+1} = \sum_{i=1}^n s_{t+1,i}$ ,  $\theta_{t+1} = \theta_0 + \bar{s}_{t+1} d_{t+1}$ .

## 7.2 Randomized Updating

**GLD (Gradient Langevin Dynamics)** (Durmus & Moulines, 2017; Dalalyan, 2017a,b)  $\theta_0 = \mathbf{0}$ ,  $\epsilon_t \sim N(0, I_{d \times d})$ ,  $\theta_{t+1} = \theta_t - \eta g_t + \sqrt{2\eta/\beta} \epsilon_t$ , where  $g_t$  is the full gradient.

**SGLD** (Welling & Teh, 2011) Same as GLD, but  $g_t$  is the average gradient over samples in small batch.

**SGFS** (Ahn et al., 2012) For total number of samples  $N$  and batch size  $B$ ,  $\gamma = \frac{B+N}{B}$ . For small batch  $\{\xi_{t,i}\}_{i=1}^B$ ,  $v_t = (1-\beta)v_{t-1} + \beta \frac{1}{n-1} \sum_{i=1}^B (\nabla f(\theta_t, \xi_{t,i}) - g_t)(\nabla f(\theta_t, \xi_{t,i}) - g_t)^\top$ ,  $\epsilon_t \sim \mathcal{N}(0, \frac{4C}{\epsilon})$ ,  $\theta_{t+1} = \theta_t + 2(\gamma N v_t + \frac{4C}{\epsilon})^{-1}(g_t + \epsilon_t)$ .

**SVRG-LD** (Xu et al., 2018) For each epoch  $s$ , first compute the full gradient  $g_s$  for  $\theta_s$ ,  $\tilde{g}_{s,t} = g_{s,t} - \nabla f(g_s, \xi_{s,t}) + g_s$ ,  $\epsilon_t \sim N(0, I_{d \times d})$ ,  $\theta_{t+1} = \theta_t - \eta \tilde{g}_{s,t} + \sqrt{2\eta/\beta} \epsilon_t$ . At the last iteration  $T$  for each epoch,  $g_s = g_{s,T}$ .

## 7.3 Optimizer Accessories

Some algorithms provide framework or “accessories” based on the update of optimizer  $\Delta_t$ , like  $-\eta_t \frac{m_t}{\sqrt{v_t + \epsilon}}$  for Adam, to seek for better properties, such as stability or more precise control.

**Drop-Muon** (Gruntkowska et al., 2025) (Based on Muon optimizer update  $\Delta_t$ ). For each iteration, freeze some layers and for other layers, update with Muon.

**SkipUpdate** (Joo et al., 2026) For the update of a base optimizer  $\Delta_t$ ,  $\tilde{m}_t \sim \text{Bernoulli}(0.5)$ ,  $\theta_{t+1} = \theta_t + 2\tilde{m}_t \Delta_t$ .

**Magma** (Joo et al., 2026) (Based on SkipUpdate)  $\tilde{s}_t = \text{Sigmoid}(\frac{\langle m_t, g_t \rangle}{\tau \|m_t\| \|g_t\|})$ , where  $m_t$  is the first-order momentum,  $s_t = 0.9s_{t-1} + 0.1\tilde{s}_t$ ,  $\theta_{t+1} = \theta_t + s_t \tilde{m}_t \Delta_t$ .

**Lookahead Optimizer** (Zhang et al., 2019) For each sub-step  $\phi_{t,i}$ ,  $i \in \{1, \dots, k\}$  with  $\phi_{t,0} = \theta_t$ , for the update of a base optimizer  $\Delta_t(\phi_{t,i-1}, \xi_{t,i})$  (learning rate included),  $\phi_{t,i} = \phi_{t,i-1} + \Delta_t(\phi_{t,i-1}, \xi_{t,i})$ . Then Update with  $\theta_{t+1} = \theta_t + \eta_t(\phi_{t,k} - \theta_t)$ .

**Layca** (Carbannelle & De Vleeschouwer, 2018) For the update of a base optimizer  $\Delta_t$ , (for each layer  $l$ )  $s_t = \Delta_t - \frac{(\Delta_t \cdot \theta_t) \theta_t}{\|\theta_t\|_2^2}$ ,  $\hat{\theta}_t = \theta_t + \eta_t^l s_t$ ,  $\theta_{t+1} = \hat{\theta}_t \frac{\|\theta_0\|_2}{\|\hat{\theta}_t\|_2}$ .

**ARO** (Gong et al., 2026)  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $R_t = \text{QR}(m_t f_t(R_{t-1}^\top m_t)^\top)$ ,  $\theta_{t+1} = \theta_t - \eta R_t f_t(R_t^\top m_t)$ , where  $f_t$  is the update function (without learning rate) for base optimizers like Adam or Sign.

**GradientStabilizer** (Huang et al., 2025b)  $R_t = \|g_t\|_2$ ,  $m_t = \beta_1 m_{t-1} + (1 - \beta_1)R_t$ ,  $v_t = \beta_2 m_{t-1} + (1 - \beta_2)R_t^2$ ,  $\tilde{g}_t = \frac{m_t g_t}{R_t \sqrt{v_t}}$ , use  $\tilde{g}_t$  for base optimizer update.

**GPA** (Defazio et al., 2025)  $y_t = \gamma_y \theta_t + (1 - \gamma_y)z_t$ , get the update of a base optimizer  $\Delta_t(y_t, \xi_t)$  with  $y_t$ , then  $z_{t+1} = (1 - \eta\lambda)z_t + \eta\Delta_t$ ,  $\theta_{t+1} = \gamma_x \theta_t + (1 - \gamma_x)z_{t+1}$

## 7.4 Reconciliation of Optimizers

**AdaGraft** (Agarwal et al., 2020) For optimizers  $\mathcal{M}, \mathcal{D}$ ,  $\theta_{t,\mathcal{M}} = \mathcal{M}(\theta_t, g_t)$ ,  $\theta_{t,\mathcal{D}} = \mathcal{D}(\theta_t, g_t)$ ,  $\theta_{t+1} = \theta_t + \frac{\|\theta_{t,\mathcal{M}} - \theta_t\|}{\|\theta_{t,\mathcal{D}} - \theta_t\| + \epsilon} \cdot (\theta_{t,\mathcal{D}} - \theta_t)$

## 7.5 Architecture-specific Optimizers

**GaLore** (Zhao et al., 2024a) (Adam for LLM layer weight matrix):  $\theta_t \in \mathbb{R}^{m \times n}$ , if  $t \bmod T = 0$   $U, S, V = \text{SVD}(g_t)$ ,  $P_t = U[:, :r]$  (low-rank projection), otherwise  $P_t = P_{t-1}$ . Then estimate the low-rank gradient  $R_t = P_t^\top g_t$ , and use Adam to optimize:  $\theta_{t+1} = \theta_t + \eta \alpha P_t \frac{m_t}{\sqrt{v_t + \epsilon}}$  with scale factor  $\alpha$

**Adam-mini** (Zhang et al., 2024) (Based on Adam) (For each parameter in parameter blocks)  $v_t = \beta_2 v_{t-1} + (1 - \beta_2) * \text{Mean}(g \odot g)$ . For Transformers, partition the parameters of embedding and output layers by tokens, partition the parameters of query and key matrices by heads, and partition the parameters of value matrices, attention projection matrices and MLP layers by output neurons.

**Adalayer** (Zhao et al., 2024b) (Based on Adam, for language model) For each layer:  $v_t = \beta_2 v_{t-1} + (1 - \beta_2) \|g_t\|_2^2 / \sqrt{p}$  with  $p$  the number of parameters in each layer,  $\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t + \epsilon}}$

**AdamC** (Based on AdamW) (Defazio, 2025) (Only for the layer immediately followed by a normalization operation including LayerNorm or BatchNorm, otherwise use AdamW):  $\theta_{t+1} = \theta_t - \eta (\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \frac{\eta}{\eta_{\max}} \lambda \theta_t)$ . (In Transformers, AdamC is applied to every linear layer except the output layer.)

**STORM-PG** (Yuan et al., 2020) (Based on Reinforce Learning tasks)  $\theta_{t+1} = \theta_t + \eta \hat{g}_t$ ,  $\hat{g}_{t+1} = (1 - \gamma)g'_t + g_t$ , where  $g_t = \frac{1}{B} \sum_{i \in \mathcal{B}} d_i(\theta_{t+1})$ ,  $g'_t = \frac{1}{B} \sum_{i \in \mathcal{B}} [\hat{g}_t - d_i^{\theta_{t+1}}(\theta_t)]$  is the gradient estimation with different parameters on small batch of trajectories  $\{\tau_i\}_{i \in \mathcal{B}}$  with size  $B$ , and  $d_i(\theta) = \sum_{h=0}^{H-1} d_{i,h}(\theta)$ ,  $d_i^{\theta'}(\theta) = \sum_{h=0}^{H-1} \frac{p(\tau_{i,h}|\theta)}{p(\tau_{i,h}|\theta')} d_{i,h}(\theta)$ , where  $d_{i,h}(\theta) = (\sum_{t=0}^h \nabla \log \pi_\theta(a_t|s_t))(\gamma^h r(s_h, a_h) - b_h)$ .

**Muon-clip (QK-Clip)** (Moonshot AI, 2025) (Based on Muon, for Attention mechanism) In each layer, for input  $\{x_i\}$ , query and key weight  $\mathbf{W}_q, \mathbf{W}_k$ ,  $S_{\max} = (\max_{ij} x_i \mathbf{W}_q) \cdot (\mathbf{x}_j \mathbf{W}_k)$ . First update all parameters using Muon, then if  $S_{\max} > \tau$ :

- MHA/MQA/GQA: multiply  $\sqrt{\tau/S_{\max}}$  to  $\mathbf{W}_q, \mathbf{W}_k$ ;
- MLA: multiply  $\sqrt{\tau/S_{\max}}$  to  $\mathbf{W}_{qc}, \mathbf{W}_{kc}$  and multiply  $\tau/S_{\max}$  to  $\mathbf{W}_{qr}$ .

**Nero** (Liu et al., 2021)

- For weight matrix,  $v_t = \beta v_{t-1} + (1 - \beta)\|g_t\|_2^2$ ,  $w_t = \theta_t - \eta \frac{\|\theta_t\|_2}{v_t} g_t$ ,  $\tilde{w}_t = w_t - \frac{1}{n} \sum_i w_{t,i}$ ,  $\theta_{t+1} = \frac{\tilde{w}_t}{\|\tilde{w}_t\|_2}$ ;
- For bias,  $v_t = \beta v_{t-1} + (1 - \beta)\|g_t\|_2^2$ ,  $\theta_{t+1} = \theta_t - \eta \frac{\sigma_b}{v_t} g_t$ , where  $\sigma_b$  is the initialization scale of the bias.

**SCALE** (Glentis et al., 2025)  $m_t = \beta m_{t-1} + (1 - \beta)g_t$  if last layer else  $m_t = g_t$ .  $\theta_{t+1} = \theta_t - \eta \mathcal{C}(m_t)$ , where  $\mathcal{C}$  is column-wise normalization.

**Teon** For similar K parameter matrices in  $\mathbb{R}^{m \times n}$ , it groups them together for optimization:  $g_t \in \mathbb{R}^{m \times n \times K}$ ,  $m_t = \mu m_{t-1} + g_t$ ,  $Z_t = \text{Reshape}(m_t, m \times (nK))$ ,  $O_t = \text{Ortho}(Z_t)$ ,  $U_t = \text{Reshape}(O_t, m \times n \times K)$ ,  $\theta_{t+1} = \theta_t - \eta \sqrt{m/n} U_t$ .

## 7.6 Weight Norm and Oracle-based Optimizers

**Linear Minimization Oracle (LMO)**: For norm-ball  $\mathcal{D} := \{x \mid \|x\| \leq \rho\}$  for some norm  $\|\cdot\|$  (Euclidean norm in default),  $\text{lmo}(s) \in \arg \min_{x \in \mathcal{D}} \langle s, x \rangle$ : for  $s \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ :

- $1 \rightarrow \text{RMS}$  (ColNorm, such as Embedding):  $\text{col}_j(s) \rightarrow -\sqrt{d_{\text{out}}} \frac{\text{col}_j(s)}{\|\text{col}_j(s)\|_2}$ ;
- $1 \rightarrow \infty$  (Sign):  $s \rightarrow -\text{sign}(s)$ ;
- $\text{RMS} \rightarrow \text{RMS}$  (Spectral, such as Linear module):  $s \rightarrow -\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}} UV^\top$  for  $U, \Sigma, V^\top = \text{SVD}(s)$ ;
- $\text{RMS} \rightarrow \infty$  (RowNorm, such as LM Head):  $\text{row}_i(s) = -\frac{1}{\sqrt{d_{\text{in}}}} \frac{\text{row}_i(s)}{\|\text{row}_i(s)\|_2}$

**Conditional Gradient Method (CG)** (Frank et al., 1956; Clarkson, 2010; Jaggi, 2013)  $\theta_{t+1} = (1 - \eta)\theta_t + \eta \cdot \text{lmo}(g_t)$

**Stochastic Conditional Gradient (SCG)** (Pethick et al., 2025)  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $\theta_{t+1} = (1 - \eta)\theta_t + \eta \cdot \text{lmo}(m_t)$  ( $\theta_{t+1} = \theta_t + \eta \cdot \text{lmo}(m_t)$  for Unconstrained SCG (uSCG)). For different base optimization method, the LMOs are different:

- Normalized SGD (Hazan et al., 2015) and Momentum Normalized SGD (Cutkosky & Mehta, 2020):  $-\rho \frac{m_t}{\|m_t\|_2}$ ;
- SignSGD (Bernstein et al., 2018) and Signum (Bernstein et al., 2018):  $-\rho \text{sign}(m_t)$ ;
- Muon (Jordan et al., 2024) (with non-Nesterov based momentum):  $-\rho UV^\top$  for  $U \Sigma V^\top = \text{SVD}(m_t)$ .

**Scion** (Pethick et al., 2025)  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $\theta_{t+1} = (1 - \eta)\theta_t + \eta \cdot \text{lmo}_{\|\cdot\|_{\alpha \rightarrow \beta}}(m_t)$  ( $\theta_{t+1} = \theta_t + \eta \cdot \text{lmo}_{\|\cdot\|_{\alpha \rightarrow \beta}}(m_t)$  for unconstrained version), where  $\|A\|_{\alpha \rightarrow \beta} = \sup_{\|z\|_{\alpha}=1} \|Az\|_{\beta}$ .

**GGNC** (Pethick et al., 2026)  $m_t = \alpha g_t + (1 - \alpha)m_{t-1}$ ,  $v_t = -\text{lmo}(m_t)$   $\tau_t = \min\{\rho, \langle m_t, v_t \rangle\}$ ,  $\theta_{t+1} = \theta_t - \eta \tau_t v_t$ .

**S<sup>3</sup>CG** (Pethick et al., 2026)  $m_t = \alpha g_t + (1 - \alpha)m_{t-1}$ ,  $v_t = \theta_t - \beta \text{lmo}(m_t)$   $\tau_t = \min\{\rho, \frac{\langle m_t, v_t \rangle}{\|v_t\|^2}\}$  or  $\tau_t = \min\{\rho, \frac{\langle m_t, v_t \rangle}{4\beta^2}\}$ ,  $\theta_{t+1} = \theta_t - \eta \tau_t v_t$ .

**RanSOM-E** (Chayti, 2026)  $m_t = (1 - \beta)(m_{t-1} + \eta_{t-1} g_{t+1}^2 d_{t-1}) + \beta g_t$ ,  $d_t = \text{lmo}(m_t)$ , sample  $s_t \sim \text{Exp}(1/\eta_t)$ ,  $\theta_{t+1} = \theta_t + s_t d_t$ .

**RanSOM-B** (Chayti, 2026)  $v_t = \text{lmo}(m_t)$ ,  $K_t = 1/\eta_t - 1$ , sample  $s_t \sim \text{Beta}(1, K_t)$ ,  $\theta_{t+1} = \theta_t + s_t(v_t - x_t)$ ,  $m_{t+1} = (1 - \beta)(m_t + \frac{1-s_t}{K_t} g_{t+1}^2 (v_t - x_t)) + \beta g_{t+1}$ .

**Gluon** (Riabinin et al., 2025)  $m_t = \beta m_{t-1} + (1 - \beta)g_t$ ,  $\theta_{t+1} = \arg \min_{\|\theta_{t+1} - \theta_t\| \leq p_t} \langle m_t, \theta_t \rangle$  ( $p_t$  can be  $\frac{\|g_t\|}{L_0 + L_1 \|g_t\|}$  or  $\frac{L_0}{(t+1)^{3/4}}$ ).

**MNGD** (Scetbon et al., 2025)  $\theta_{t+1} = \theta_t - \eta \arg \max_{\|z\|_d=1} \langle g_t, z \rangle$  for  $d$ -norms.

**SinkGD** (Scetbon et al., 2025)  $X_{t,0} = g_t$  For  $L$  iterations,  $X'_{t,k} = \sqrt{n}Q(X_{t,k-1})^{-1}X_{t,k-1}$ ,  $X_{t,k+1} = \sqrt{m}X'_{t,k}R(X'_{t,k})^{-1}$ , where  $Q(\cdot)$  and  $R(\cdot)$  are positive coordinate-wise and diagonal matrices so that  $X = QAR$  as described in Sinkhorn theorem (Sinkhorn & Knopp, 1967).  $\theta_{t+1} = \theta_t - \eta X_{t,L}$

## 7.7 Something Beyond Optimizers

In the previous sections, I did not emphasize the changing of the learning rate  $\eta$ . In this section, I collected some interesting research related to the changing rule of learning rate (Scheduler).

### 7.7.1 Scheduler

**Warmup-Stable-Decay (WSD) Scheduler** (Hu et al., 2024) Different from the Cosine Learning Rate Scheduler, in MiniCPM, the researchers utilized WSD scheduler (linear warmup-stable learning rate-decay). It is widely used in industry because it can support continual training, and decaying schedule in the end is also compatible for downstream tasks fine-tuning.

For intermediate checkpoints, WSD performs decay schedule and starts from the state before decay when continuing training. However, in **WSD-S** (Wen et al., 2024), the researchers find that the dispose of training in decay phase is not necessary, and just starts from the state after decay with the max learning rate is fine.

### 7.7.2 Schedule Refinement

**Schedule Refinement** (Defazio et al., 2023) The researchers perform comprehensive evaluation of learning rate schedules, give proofs on the convergence of some common optimization approaches, and proposed some important conclusions and algorithms:

- Warm-up followed by linear decay is the best overall non-adaptive schedule, outperforming cosine decay.
- **Schedule Refinement for SGD**  $\hat{g}_t = \text{Median-filter}(\|g_t\|, \text{width} = \tau T, \text{pad} = (\text{nearest}, \text{reflect}))$ ,  $w_t = \hat{g}_t^{-2}$ ,  $\eta'_t = w_t \sum_{p=t+1}^T w_p$ ,  $\eta_t = \eta'_t / \max_p(\eta'_p)$ .
- **Schedule Refinement for Adam**  $\hat{g}_t = \text{Median-filter}(\|\sum_{i=1}^d \frac{g_{t,i}^2}{\sqrt{v_{t,i}}}\|, \text{width} = \tau T, \text{pad} = (\text{nearest}, \text{reflect}))$ ,  $w_t = \hat{g}_t^{-1}$ ,  $\eta'_t = w_t \sum_{p=t+1}^T w_p$ ,  $\eta_t = \eta'_t / \max_p(\eta'_p)$ .

### 7.7.3 “Scaling Law” of Learning Rate

**$\mu$ P and Tensor Program series** (Yang et al., 2022) In Tensor Program series, the researchers led by Greg Yang<sup>1</sup> provides theoretical foundation for the “Scaling Law” of hyper-parameter to the model size. In Tensor Program V ( $\mu$ P), they researched how the best hyper-parameters (including initialization variances and learning rates for different optimizers of different components in deep learning models) change with respect to the size of these components.

## 8 Acknowledgement

Thank Prof. Quanquan Gu, and other students in UCLA AGI Lab including Huizhuo Yuan. Thank former colleagues from ByteDance Inc. and Moonshot Inc. including Jianlin Su for his blogs at <https://kexue.fm/>.

## References

Naman Agarwal, Rohan Anil, Elad Hazan, Tomer Koren, and Cyril Zhang. Disentangling adaptive gradient methods from learning rates. *arXiv preprint arXiv:2002.11803*, 2020.

Kwangjun Ahn, Noah Amsel, and John Langford. Dion2: A simple method to shrink matrix in muon. *arXiv preprint arXiv:2512.16928*, 2025a.

<sup>1</sup><https://thegregyang.com/>

- Kwangjun Ahn, Byron Xu, Natalie Abreu, and John Langford. Dion: Distributed orthonormalized updates. *arXiv preprint arXiv:2504.05295*, 2025b.
- Sungjin Ahn, Anoop Korattikara Balan, and Max Welling. Bayesian posterior sampling via stochastic gradient fisher scoring. In *Proceedings of the 29th International Conference on Machine Learning, ICML 2012, Edinburgh, Scotland, UK, June 26 - July 1, 2012*. icml.cc / Omnipress, 2012. URL <http://icml.cc/2012/papers/782.pdf>.
- Shun-Ichi Amari. Natural gradient works efficiently in learning. *Neural computation*, 10(2):251–276, 1998.
- Noah Amsel, David Persson, Christopher Musco, and Robert Gower. The polar express: Optimal matrix sign methods and their application to the muon algorithm. *arXiv preprint arXiv:2505.16932*, 2025.
- Kang An, Yuxing Liu, Rui Pan, Shiqian Ma, Donald Goldfarb, and Tong Zhang. Asgo: Adaptive structured gradient optimization. *arXiv preprint arXiv:2503.20762*, 2025.
- Larry Armijo. Minimization of functions having lipschitz continuous first partial derivatives. *Pacific Journal of mathematics*, 16(1):1–3, 1966.
- Sue Becker. Improving the convergence of backpropagation learning with second order method. In *Proceedings of the 1988 Connectionist Models Summer School, San Mateo, CA*. Morgan Kaufmann, 1988.
- Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. Nested learning: The illusion of deep learning architectures. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Frederik Benzing. Gradient descent on neurons and its link to approximate second-order optimization. In *International conference on machine learning*, pp. 1817–1853. PMLR, 2022.
- Jeremy Bernstein and Laker Newhouse. Old optimizer, new norm: An anthology. *arXiv preprint arXiv:2409.20325*, 2024.
- Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Animashree Anandkumar. signsgd: Compressed optimisation for non-convex problems. In *International Conference on Machine Learning*, pp. 560–569. PMLR, 2018.
- Jeremy Bernstein, Arash Vahdat, Yisong Yue, and Ming-Yu Liu. On the distance between two neural networks and the stability of learning. *Advances in Neural Information Processing Systems*, 33: 21370–21381, 2020.
- Yang Cao, Xiaoyu Li, and Zhao Song. Grams: Gradient descent with adaptive momentum scaling. *arXiv preprint arXiv:2412.17107*, 2024.
- Simon Carbonnelle and Christophe De Vleeschouwer. Layer rotation: a surprisingly powerful indicator of generalization in deep networks? *arXiv preprint arXiv:1806.01603*, 2018.
- David Carlson, Volkan Cevher, and Lawrence Carin. Stochastic Spectral Descent for Restricted Boltzmann Machines. In Guy Lebanon and S. V. N. Vishwanathan (eds.), *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics*, volume 38 of *Proceedings of Machine Learning Research*, pp. 111–119, San Diego, California, USA, 09–12 May 2015. PMLR. URL <https://proceedings.mlr.press/v38/carlson15.html>.
- Da Chang and Gaozhao Yuan. Mgup: A momentum-gradient alignment update policy for stochastic optimization. *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Da Chang, Yongxiang Liu, and Ganzhao Yuan. On the convergence of muon and beyond. *arXiv preprint arXiv:2509.15816*, 2025.
- El Mahdi Chayti. Ransom: Second-order momentum with randomized scaling for constrained and unconstrained optimization. *arXiv preprint arXiv:2602.06824*, 2026.

- Jinghui Chen and Quanquan Gu. Padam: Closing the generalization gap of adaptive gradient methods in training deep neural networks. 2018.
- Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.
- Xiangyi Chen, Sijia Liu, Ruoyu Sun, and Mingyi Hong. On the convergence of a class of adam-type algorithms for non-convex optimization. *arXiv preprint arXiv:1808.02941*, 2018.
- Kenneth L Clarkson. Coresets, sparse greedy approximation, and the frank-wolfe algorithm. *ACM Transactions on Algorithms (TALG)*, 6(4):1–30, 2010.
- Michael Crawshaw, Chirag Modi, Mingrui Liu, and Robert M Gower. An exploration of non-euclidean gradient descent: Muon and its many variants. *arXiv preprint arXiv:2510.09827*, 2025.
- Ashok Cutkosky and Harsh Mehta. Momentum improves normalized sgd. In *International conference on machine learning*, pp. 2260–2268. PMLR, 2020.
- Ashok Cutkosky, Aaron Defazio, and Harsh Mehta. Mechanic: A learning rate tuner. *Advances in neural information processing systems*, 36:47828–47848, 2023.
- Arnak Dalalyan. Further and stronger analogy between sampling and optimization: Langevin monte carlo and gradient descent. In *Conference on Learning Theory*, pp. 678–689. PMLR, 2017a.
- Arnak S Dalalyan. Theoretical guarantees for approximate sampling from smooth and log-concave densities. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 79(3):651–676, 2017b.
- Aaron Defazio. Why gradients rapidly increase near the end of training. *arXiv preprint arXiv:2506.02285*, 2025.
- Aaron Defazio and Samy Jelassi. A momentumized, adaptive, dual averaged gradient method. *Journal of Machine Learning Research*, 23(144):1–34, 2022.
- Aaron Defazio and Konstantin Mishchenko. Learning-rate-free learning by d-adaptation. In *International Conference on Machine Learning*, pp. 7449–7479. PMLR, 2023.
- Aaron Defazio, Francis Bach, and Simon Lacoste-Julien. Saga: A fast incremental gradient method with support for non-strongly convex composite objectives. *Advances in neural information processing systems*, 27, 2014.
- Aaron Defazio, Ashok Cutkosky, Harsh Mehta, and Konstantin Mishchenko. When, why and how much? adaptive learning rate scheduling by refinement. *arXiv preprint arXiv:2310.07831*, 2023.
- Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled. *arXiv preprint arXiv:2405.15682*, 2024.
- Aaron Defazio, Konstantin Mishchenko, Parameswaran Raman, Hao-Jun Michael Shi, and Lin Xiao. Smoothing diloco with primal averaging for faster training of llms. *arXiv preprint arXiv:2512.17131*, 2025.
- Shenyang Deng, Zhuoli Ouyang, Tianyu Pang, Zihang Liu, Ruochen Jin, Shuhua Yu, and Yaoqing Yang. Rmnp: Row-momentum normalized preconditioning for scalable matrix-based optimization. *arXiv preprint arXiv:2603.20527*, 2026.
- Jianbang Ding, Xuancheng Ren, Ruixuan Luo, and Xu Sun. An adaptive and momental bound method for stochastic learning. *arXiv preprint arXiv:1910.12249*, 2019.
- Timothy Dozat. Incorporating nesterov momentum into adam. 2016.
- Zhehang Du and Weijie Su. The newton-muon optimizer. *arXiv preprint arXiv:2604.01472*, 2026.

- Shiv Ram Dubey, Soumendu Chakraborty, Swalpa Kumar Roy, Snehasis Mukherjee, Satish Kumar Singh, and Bidyut Baran Chaudhuri. diffgrad: an optimization method for convolutional neural networks. *IEEE transactions on neural networks and learning systems*, 31(11):4500–4511, 2019.
- John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011.
- Alain Durmus and Éric Moulines. Nonasymptotic convergence analysis for the unadjusted langevin algorithm. *The Annals of Applied Probability*, pp. 1551–1587, 2017.
- Sai Surya Duvvuri, Fnu Devvrit, Rohan Anil, Cho-Jui Hsieh, and Inderjit S Dhillon. Combining axes preconditioners through kronecker approximation for deep learning. In *The Twelfth International Conference on Learning Representations*, 2024.
- Cong Fang, Chris Junchi Li, Zhouchen Lin, and Tong Zhang. Spider: Near-optimal non-convex optimization via stochastic path-integrated differential estimator. *Advances in neural information processing systems*, 31, 2018.
- Roger Fletcher. *Practical Methods of Optimization*. John Wiley & Sons, New York, 2 edition, 1987. ISBN 978-0-471-91547-8.
- Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-aware minimization for efficiently improving generalization. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*, 2021.
- Marguerite Frank, Philip Wolfe, et al. An algorithm for quadratic programming. *Naval research logistics quarterly*, 3(1-2):95–110, 1956.
- Boris Ginsburg, Patrice Castonguay, Oleksii Hrinchuk, Oleksii Kuchaiev, Vitaly Lavrukhin, Ryan Leary, Jason Li, Huyen Nguyen, Yang Zhang, and Jonathan M Cohen. Stochastic gradient methods with layer-wise adaptive moments for training of deep networks. *arXiv preprint arXiv:1905.11286*, 2019.
- Athanasios Glentis, Jiaxiang Li, Andi Han, and Mingyi Hong. A minimalist optimizer design for llm pretraining. In *ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models*, 2025.
- Wenbo Gong, Javier Zazo, Qijun Luo, Puqian Wang, James Hensman, and Chao Ma. Aro: A new lens on matrix optimization for large models. *arXiv preprint arXiv:2602.09006*, 2026.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Roger Grosse and James Martens. A kronecker-factored approximate fisher matrix for convolution layers. In *International Conference on Machine Learning*, pp. 573–582. PMLR, 2016.
- Marcus Grum. Learning representations by crystallized back-propagating errors. In Leszek Rutkowski, Rafal Scherer, Marcin Korytkowski, Witold Pedrycz, Ryszard Tadeusiewicz, and Jacek M. Zurada (eds.), *Artificial Intelligence and Soft Computing - 22nd International Conference, ICAISC 2023, Zakopane, Poland, June 18-22, 2023, Proceedings, Part I*, volume 14125 of *Lecture Notes in Computer Science*, pp. 78–100. Springer, 2023. doi: 10.1007/978-3-031-42505-9\_8. URL [https://doi.org/10.1007/978-3-031-42505-9\\_8](https://doi.org/10.1007/978-3-031-42505-9_8).
- Kaja Gruntkowska, Yassine Maziane, Zheng Qu, and Peter Richtárik. Drop-muon: Update less, converge faster. *arXiv preprint arXiv:2510.02239*, 2025.
- Yufei Gu and Zeke Xie. Mano: Restriking manifold optimization for llm training. *arXiv preprint arXiv:2601.23000*, 2026.
- Vineet Gupta, Tomer Koren, and Yoram Singer. A unified approach to adaptive regularization in online and stochastic optimization. *arXiv preprint arXiv:1706.06569*, 2017.
- Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In *International Conference on Machine Learning*, pp. 1842–1850. PMLR, 2018.



- Elad Hazan, Amit Agarwal, and Satyen Kale. Logarithmic regret algorithms for online convex optimization. *Machine Learning*, 69(2):169–192, 2007.
- Elad Hazan, Kfir Levy, and Shai Shalev-Shwartz. Beyond convexity: Stochastic quasi-convex optimization. *Advances in neural information processing systems*, 28, 2015.
- Chuan He, Zhanwang Deng, and Zhaosong Lu. Low-rank orthogonalization for large-scale matrix optimization with applications to foundation model training. *arXiv preprint arXiv:2509.11983*, 2025a.
- Wei He, Kai Han, Hang Zhou, Hanting Chen, Zhicheng Liu, Xinghao Chen, and Yunhe Wang. Root: Robust orthogonalized optimizer for neural network training. *arXiv preprint arXiv:2511.20626*, 2025b.
- Byeongho Heo, Sanghyuk Chun, Seong Joon Oh, Dongyoon Han, Sangdoo Yun, Gyuwan Kim, Youngjung Uh, and Jung-Woo Ha. Adamp: Slowing down the slowdown for momentum optimizers on scale-invariant weights. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*, 2021.
- Nicholas J Higham. Functions of matrices. society for industrial and applied mathematics, 2008.
- Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.
- Feihu Huang, Junyi Li, and Heng Huang. Super-adam: faster and universal framework of adaptive gradients. *Advances in Neural Information Processing Systems*, 34:9074–9085, 2021.
- Feihu Huang, Yuning Luo, and Songcan Chen. Limuon: Light and fast muon optimizer for large models. *arXiv preprint arXiv:2509.14562*, 2025a.
- Tianjin Huang, Haotian Hu, Zhenyu Zhang, Gaojie Jin, Xiang Li, Li Shen, Tianlong Chen, Lu Liu, Qingsong Wen, Zhangyang Wang, et al. Stable-spam: How to train in 4-bit more stably than 16-bit adam. In *First Workshop on Scalable Optimization for Efficient and Adaptive Foundation Models*, 2025b.
- Tianjin Huang, Ziquan Zhu, Gaojie Jin, Lu Liu, Zhangyang Wang, and Shiwei Liu. Spam: Spike-aware adam with momentum reset for stable llm training. *arXiv preprint arXiv:2501.06842*, 2025c.
- Michael F Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics-Simulation and Computation*, 18(3):1059–1076, 1989.
- Maor Ivgi, Oliver Hinder, and Yair Carmon. Dog is sgd’s best friend: A parameter-free dynamic step size schedule. In *International conference on machine learning*, pp. 14465–14499. PMLR, 2023.
- Martin Jaggi. Revisiting frank-wolfe: Projection-free sparse convex optimization. In *International conference on machine learning*, pp. 427–435. PMLR, 2013.
- Majid Jahani, Sergey Rusakov, Zheng Shi, Peter Richtárik, Michael W Mahoney, and Martin Takac. Doubly adaptive scaled algorithm for machine learning using second-order information. In *International Conference on Learning Representations*.
- Tao Jiang and Lin Xiao. Stochastic approximation with block coordinate optimal stepsizes. *arXiv preprint arXiv:2507.08963*, 2025.
- Xiaowen Jiang and Sebastian U Stich. Adaptive sgd with polyak stepsize and line-search: Robust convergence and variance reduction. *Advances in Neural Information Processing Systems*, 36, 2024.
- Rie Johnson and Tong Zhang. Accelerating stochastic gradient descent using predictive variance reduction. *Advances in neural information processing systems*, 26, 2013.

- Taejong Joo, Wenhan Xia, Cheolmin Kim, Ming Zhang, and Eugene Je. On surprising effectiveness of masking updates in adaptive optimizers. *arXiv preprint arXiv:2602.15322*, 2026.
- Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cecista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Ali Kavis, Stratis Skoulakis, Kimon Antonakopoulos, Leello Tadesse Dadi, and Volkan Cevher. Adaptive stochastic variance reduction for non-convex finite-sum minimization. *Advances in Neural Information Processing Systems*, 35:23524–23538, 2022.
- Nitish Shirish Keskar and Richard Socher. Improving generalization performance by switching from adam to sgd. *arXiv preprint arXiv:1712.07628*, 2017.
- Ahmed Khaled, Konstantin Mishchenko, and Chi Jin. Dowg unleashed: An efficient universal parameter-free gradient descent method. *Advances in Neural Information Processing Systems*, 36:6748–6769, 2023.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun (eds.), *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.
- Jungmin Kwon, Jeongseop Kim, Hyunseo Park, and In Kwon Choi. Asam: Adaptive sharpness-aware minimization for scale-invariant learning of deep neural networks. In *International Conference on Machine Learning*, pp. 5905–5914. PMLR, 2021.
- Tim Tsz-Kit Lau, Qi Long, and Weijie Su. Polargrad: A class of matrix-gradient optimizers from a unifying preconditioning perspective. *arXiv preprint arXiv:2505.21799*, 2025.
- Kfir Levy, Ali Kavis, and Volkan Cevher. Storm+: Fully adaptive sgd with recursive momentum for nonconvex optimization. *Advances in Neural Information Processing Systems*, 34:20571–20582, 2021.
- Chris Junchi Li, Wenlong Mou, Martin Wainwright, and Michael Jordan. Root-sgd: Sharp nonasymptotics and asymptotic efficiency in a single algorithm. In *Conference on Learning Theory*, pp. 909–981. PMLR, 2022.
- Haochuan Li, Alexander Rakhlin, and Ali Jadbabaie. Convergence of adam under relaxed assumptions. *Advances in Neural Information Processing Systems*, 36:52166–52196, 2023.
- Jingru Li, Yibo Fan, and Huan Li. Variance-adaptive muon: Accelerating llm pretraining with nsr-modulated and variance-scaled momentum. *arXiv preprint arXiv:2601.14603*, 2026.
- Wenjie Li, Zhaoyang Zhang, Xinjiang Wang, and Ping Luo. Adax: Adaptive gradient descent with exponential long term memory. *arXiv preprint arXiv:2004.09740*, 2020.
- Zichong Li, Liming Liu, Chen Liang, Weizhu Chen, and Tuo Zhao. Normuon: Making muon more efficient and scalable. *arXiv preprint arXiv:2510.05491*, 2025.
- Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious optimizers: Improving training with one line of code. *arXiv preprint arXiv:2411.16085*, 2024.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024a.
- Dong C Liu and Jorge Nocedal. On the limited memory bfgs method for large scale optimization. *Mathematical programming*, 45(1):503–528, 1989.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- Hong Liu, Zhiyuan Li, David Leo Wright Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*, 2024b.

- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025a.
- Liming Liu, Zhenghao Xu, Zixuan Zhang, Hao Kang, Zichong Li, Chen Liang, Weizhu Chen, and Tuo Zhao. Cosmos: A hybrid adaptive optimizer for memory-efficient training of llms. *arXiv preprint arXiv:2502.17410*, 2025b.
- Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. On the variance of the adaptive learning rate and beyond. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*, 2020a.
- Mingrui Liu, Wei Zhang, Francesco Orabona, and Tianbao Yang. Adam<sup>+</sup>: A stochastic method with adaptive variance reduction. *arXiv preprint arXiv:2011.11985*, 2020b.
- Yang Liu, Jeremy Bernstein, Markus Meister, and Yisong Yue. Learning by turning: Neural architecture aware optimisation. In *International conference on machine learning*, pp. 6748–6758. PMLR, 2021.
- Yifeng Liu, Angela Yuan, and Quanquan Gu. Mars-m: When variance reduction meets matrices. *arXiv preprint arXiv:2510.21800*, 2025c.
- Yong Liu, Siqi Mai, Xiangning Chen, Cho-Jui Hsieh, and Yang You. Towards efficient and scalable sharpness-aware minimization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12360–12370, 2022.
- Ilya Loshchilov. Weight norm control. *arXiv preprint arXiv:2311.11446*, 2023.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*, 2019.
- Liangchen Luo, Yuanhao Xiong, Yan Liu, and Xu Sun. Adaptive gradient methods with dynamic bound of learning rate. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*, 2019.
- Yang Luo, Xiaozhe Ren, Zangwei Zheng, Zhuo Jiang, Xin Jiang, and Yang You. Came: Confidence-guided adaptive memory efficient optimization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 4442–4453, 2023.
- Chao Ma, Wenbo Gong, Meyer Scetbon, and Edward Meeds. Swan: Sgd with normalization and whitening enables stateless llm training. *arXiv preprint arXiv:2412.13148*, 2024.
- Jerry Ma and Denis Yarats. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2018.
- Xuezhe Ma. Apollo: An adaptive parameter-wise diagonal quasi-newton method for nonconvex stochastic optimization. *arXiv preprint arXiv:2009.13586*, 2020.
- Pouria Mahdavinia and Mehrdad Mahdavi. Low-rank momentum factorization for memory efficient training. *Transactions on Machine Learning Research*, 2025, 2025.
- James Martens and Roger Grosse. Optimizing neural networks with kronecker-factored approximate curvature. In *International conference on machine learning*, pp. 2408–2417. PMLR, 2015.
- James Martens et al. Deep learning via hessian-free optimization. In *Icml*, volume 27, pp. 735–742, 2010.
- Moonshot AI. Kimi K2: Open Agentic Intelligence, 2025. URL <https://moonshotai.github.io/Kimi-K2/>. Accessed: 2025-07-17.
- Yurii Nesterov. A method for solving the convex programming problem with convergence rate  $O(1/k^2)$ . In *Dokl akad nauk Sssr*, volume 269, pp. 543, 1983.

- Lam M Nguyen, Jie Liu, Katya Scheinberg, and Martin Takáč. Sarah: A novel method for machine learning problems using stochastic recursive gradient. In *International conference on machine learning*, pp. 2613–2621. PMLR, 2017.
- Son Nguyen, Bo Liu, Lizhang Chen, and Qiang Liu. Improving adaptive moment optimization via preconditioner diagonalization. *arXiv preprint arXiv:2502.07488*, 2025.
- Jorge Nocedal and Stephen J. Wright. Numerical optimization. Springer, New York, NY, USA, 2e edition, 2006.
- Matteo Pagliardini, Pierre Ablin, and David Grangier. The ademamix optimizer: Better, faster, older. *arXiv preprint arXiv:2409.03137*, 2024.
- Rui Pan, Yang Luo, Yuxing Liu, Yang You, and Tong Zhang. Unbiased gradient low-rank projection. *arXiv preprint arXiv:2510.17802*, 2025.
- Tianyu Pang, Yujie Fang, Zihang Liu, Shenyang Deng, Lei Hsiung, Shuhua Yu, and Yaoqing Yang. Htmuon: Improving muon via heavy-tailed spectral correction. *arXiv preprint arXiv:2603.10067*, 2026.
- Razvan Pascanu and Yoshua Bengio. Revisiting natural gradient for deep networks. In Yoshua Bengio and Yann LeCun (eds.), *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, 2014.
- Thomas Pethick, Wanyun Xie, Kimon Antonakopoulos, Zhenyu Zhu, Antonio Silveti-Falls, and Volkan Cevher. Training deep learning models with norm-constrained lmos. *arXiv preprint arXiv:2502.07529*, 2025.
- Thomas Pethick, Wanyun Xie, Mete Erdogan, Kimon Antonakopoulos, Antonio Silveti-Falls, and Volkan Cevher. Generalized gradient norm clipping & non-euclidean  $(l_0, l_1)$ -smoothness. *Advances in Neural Information Processing Systems*, 38:21170–21208, 2026.
- Boris T Polyak and Anatoli B Juditsky. Acceleration of stochastic approximation by averaging. *SIAM journal on control and optimization*, 30(4):838–855, 1992.
- Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 1(2):3, 2022.
- Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of adam and beyond. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*, 2018.
- Tilde Research. Aurora: Leverage-aware updates for rectangular matrices. X (formerly Twitter), May 2026. URL <https://x.com/tilderresearch/status/2052797780595499325>. Posted by @tilderresearch.
- Artem Riabinin, Egor Shulgin, Kaja Grutkowska, and Peter Richtárik. Gluon: Making muon & scion great again!(bridging theory and practice of lmo-based optimizers for llms). *arXiv preprint arXiv:2505.13416*, 2025.
- M. Riedmiller and H. Braun. A direct adaptive method for faster backpropagation learning: the rprop algorithm. In *IEEE International Conference on Neural Networks*, pp. 586–591 vol.1, 1993. doi: 10.1109/ICNN.1993.298623.
- Nicolas Roux, Mark Schmidt, and Francis Bach. A stochastic gradient method with an exponential convergence rate for finite training sets. *Advances in neural information processing systems*, 25, 2012.
- Meyer Scetbon, Chao Ma, Wenbo Gong, and Edward Meeds. Gradient multi-normalization for stateless and scalable llm training. *arXiv preprint arXiv:2502.06742*, 2025.
- Tom Schaul, Sixin Zhang, and Yann LeCun. No more pesky learning rates. In *International conference on machine learning*, pp. 343–351. PMLR, 2013.

- Shai Shalev-Shwartz and Tong Zhang. Stochastic dual coordinate ascent methods for regularized loss minimization. *Journal of Machine Learning Research*, 14(1), 2013.
- Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*, pp. 4596–4604. PMLR, 2018.
- Kexuan Shi, Hanxuan Li, Zeju Qiu, Yandong Wen, Simon Buchholz, and Weiyang Liu. Pion: A spectrum-preserving optimizer via orthogonal equivalence transformation. *arXiv preprint arXiv:2605.12492*, 2026.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Chongjie Si, Debing Zhang, and Wei Shen. Adamuon: Adaptive muon optimizer. *arXiv preprint arXiv:2507.11005*, 2025.
- Richard Sinkhorn and Paul Knopp. Concerning nonnegative matrices and doubly stochastic matrices. *Pacific Journal of Mathematics*, 21(2):343–348, 1967.
- Jianlin Su. Tiger: A tight-fisted optimizer. <https://github.com/bojone/tiger>, 2023.
- Shohei Taniguchi, Keno Harada, Gouki Minegishi, Yuta Oshima, Seong Cheol Jeong, Go Nagahara, Tomoshi Iiyama, Masahiro Suzuki, Yusuke Iwasawa, and Yutaka Matsuo. Adopt: Modified adam can converge with any  $\beta_2$  with the optimal rate. *Advances in Neural Information Processing Systems*, 37:72438–72474, 2024.
- Yuanzhe Tao, Huizhuo Yuan, Xun Zhou, Yuan Cao, and Quanquan Gu. Towards simple and provable parameter-free adaptive gradient methods, 2024. URL <https://arxiv.org/abs/2412.19444>.
- Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- Ran Tian and Ankur P Parikh. Amos: An adam-style optimizer with adaptive weight decay towards model-oriented scale. *arXiv preprint arXiv:2210.11693*, 2022.
- Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural networks for machine learning*, 4(2):26–31, 2012.
- Kristi Topollai and Anna Choromanska. Adaptive memory momentum via a model-based framework for deep learning optimization. *arXiv preprint arXiv:2510.04988*, 2025.
- Phuong Thi Tran et al. On the convergence proof of amsgrad and a new version. *IEEE Access*, 7: 61706–61716, 2019.
- Quoc Tran-Dinh, Nhan H Pham, Dzung T Phan, and Lam M Nguyen. Hybrid stochastic gradient descent algorithms for stochastic nonconvex optimization. *arXiv preprint arXiv:1905.05920*, 2019.
- Mark Tuddenham, Adam Prügel-Bennett, and Jonathan Hare. Orthogonalising gradients to speed up neural network optimisation. *arXiv preprint arXiv:2202.07052*, 2022.
- Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. Soap: Improving and stabilizing shampoo using adam. *arXiv preprint arXiv:2409.11321*, 2024.
- Junjie Wang, Pan Zhou, Yiming Dong, Huan Li, Jia Li, Xun Zhou, Qicheng Lao, Cong Fang, and Zhouchen Lin. Conda: Column-normalized adam for training large language models faster. *arXiv preprint arXiv:2509.24218*, 2025a.
- Mingze Wang, Jinbo Wang, Jiaqi Zhang, Wei Wang, Peng Pei, Xunliang Cai, Lei Wu, et al. Gradpower: Powering gradients for faster language model pre-training. *arXiv preprint arXiv:2505.24275*, 2025b.

- Shaowen Wang, Anan Liu, Jian Xiao, Huan Liu, Yuekui Yang, Cong Xu, Qianqian Pu, Suncong Zheng, Wei Zhang, Di Wang, et al. Cadam: Confidence-based optimization for online learning. *arXiv preprint arXiv:2411.19647*, 2024.
- Shuche Wang, Fengzhuo Zhang, Jiayang Li, Cunxiao Du, Chao Du, Tianyu Pang, Zhuoran Yang, Mingyi Hong, and Vincent YF Tan. Muon outperforms adam in tail-end associative memory learning. *arXiv preprint arXiv:2509.26030*, 2025c.
- Zhe Wang, Kaiyi Ji, Yi Zhou, Yingbin Liang, and Vahid Tarokh. Spiderboost and momentum: Faster variance reduction algorithms. *Advances in Neural Information Processing Systems*, 32, 2019.
- Rachel Ward, Xiaoxia Wu, and Leon Bottou. Adagrad stepsizes: Sharp convergence over nonconvex landscapes. *Journal of Machine Learning Research*, 21(219):1–30, 2020.
- Max Welling and Yee W Teh. Bayesian learning via stochastic gradient langevin dynamics. In *Proceedings of the 28th international conference on machine learning (ICML-11)*, pp. 681–688. Citeseer, 2011.
- Kaiyue Wen, Zhiyuan Li, Jason Wang, David Hall, Percy Liang, and Tengyu Ma. Understanding warmup-stable-decay learning rates: A river valley loss landscape perspective. *arXiv preprint arXiv:2410.05192*, 2024.
- Kaiyue Wen, Xingyu Dang, Kaifeng Lyu, Tengyu Ma, and Percy Liang. Fantastic pretraining optimizers and where to find them 2.1: Hyperball optimization, 12 2025. URL <https://tinyurl.com/muonh>.
- Tian Xie, Haoming Luo, Haoyu Tang, Yiwen Hu, Jason Klein Liu, Qingnan Ren, Yang Wang, Wayne Xin Zhao, Rui Yan, Bing Su, Chong Luo, and Baining Guo. Controlled llm training on spectral sphere, 2026. URL <https://arxiv.org/abs/2601.08393>.
- Xingyu Xie, Pan Zhou, Huan Li, Zhouchen Lin, and Shuicheng Yan. Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- Zeke Xie, Xinrui Wang, Huishuai Zhang, Issei Sato, and Masashi Sugiyama. Adaptive inertia: Disentangling the effects of adaptive learning rate and momentum. In *International conference on machine learning*, pp. 24430–24459. PMLR, 2022.
- Pan Xu, Jinghui Chen, Difan Zou, and Quanquan Gu. Global convergence of langevin dynamics based algorithms for nonconvex optimization. *Advances in Neural Information Processing Systems*, 31, 2018.
- Greg Yang, Edward J Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. *arXiv preprint arXiv:2203.03466*, 2022.
- Yujie Yang. Prism: Structured optimization via anisotropic spectral shaping. *arXiv preprint arXiv:2602.03096*, 2026.
- Zhewei Yao, Amir Gholami, Sheng Shen, Mustafa Mustafa, Kurt Keutzer, and Michael Mahoney. Adahessian: An adaptive second order optimizer for machine learning. In *proceedings of the AAAI conference on artificial intelligence*, volume 35, pp. 10665–10673, 2021.
- Yida Yin, Zhiqiu Xu, Zhiyuan Li, Trevor Darrell, and Zhuang Liu. A coefficient makes svrg effective. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Yang You, Igor Gitman, and Boris Ginsburg. Large batch training of convolutional networks. *arXiv preprint arXiv:1708.03888*, 2017.
- Yang You, Jing Li, Sashank J. Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, Kurt Keutzer, and Cho-Jui Hsieh. Large batch optimization for deep learning: Training BERT in 76 minutes. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*, 2020.

- Huizhuo Yuan, Xiangru Lian, Ji Liu, and Yuren Zhou. Stochastic recursive momentum for policy gradient methods. *arXiv preprint arXiv:2003.04302*, 2020.
- Huizhuo Yuan, Yifeng Liu, Shuang Wu, Quanquan Gu, et al. Mars: Unleashing the power of variance reduction for training large models. In *Forty-second International Conference on Machine Learning*, 2025.
- Jinghui Yuan, Jiaxuan Zou, Shuo Wang, Yong Liu, and Feiping Nie. Nora: Normalized orthogonal row alignment for scalable matrix optimizer. *arXiv preprint arXiv:2605.03769*, 2026.
- Manzil Zaheer, Sashank Reddi, Devendra Sachan, Satyen Kale, and Sanjiv Kumar. Adaptive methods for nonconvex optimization. *Advances in neural information processing systems*, 31, 2018.
- Matthew D Zeiler. Adadelata: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012.
- Aston Zhang, Zachary C Lipton, Mu Li, and Alexander J Smola. Dive into deep learning. *arXiv preprint arXiv:2106.11342*, pp. 505, 2021.
- Jingzhao Zhang, Sai Praneeth Karimireddy, Andreas Veit, Seungyeon Kim, Sashank Reddi, Sanjiv Kumar, and Suvrit Sra. Why are adaptive methods good for attention models? *Advances in Neural Information Processing Systems*, 33:15383–15393, 2020.
- Michael Zhang, James Lucas, Jimmy Ba, and Geoffrey E Hinton. Lookahead optimizer: k steps forward, 1 step back. *Advances in neural information processing systems*, 32, 2019.
- Minxin Zhang, Yuxuan Liu, and Hayden Scheaffer. Adam improves muon: Adaptive moment estimation with orthogonalized momentum. *arXiv preprint arXiv:2602.17080*, 2026a.
- Ruijie Zhang, Yequan Zhao, Ziyue Liu, Zhengyang Wang, and Zheng Zhang. Muon+: Towards better muon via one additional normalization step. *arXiv preprint arXiv:2602.21545*, 2026b.
- Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Yinyu Ye, Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use fewer learning rates to gain more. *arXiv preprint arXiv:2406.16793*, 2024.
- Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. Galore: Memory-efficient LLM training by gradient low-rank projection. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, 2024a.
- Rosie Zhao, Depen Morwani, David Brandfonbrener, Nikhil Vyas, and Sham Kakade. Deconstructing what makes a good optimizer for language models. *arXiv preprint arXiv:2407.07972*, 2024b.
- Beitong Zhou, Jun Liu, Weigao Sun, Ruijuan Chen, Claire J Tomlin, and Ye Yuan. pbsgd: Powered stochastic gradient descent methods for accelerated non-convex optimization. In *IJCAI*, pp. 3258–3266, 2020a.
- Dongruo Zhou, Pan Xu, and Quanquan Gu. Stochastic nested variance reduction for nonconvex optimization. *Journal of machine learning research*, 21(103):1–63, 2020b.
- Hanqing Zhu, Zhenyu Zhang, Wenyan Cong, Xi Liu, Sem Park, Vikas Chandra, Bo Long, David Z Pan, Zhangyang Wang, and Jinwon Lee. Apollo: Sgd-like memory, adamw-level performance. *Proceedings of Machine Learning and Systems*, 7, 2025.
- Juntang Zhuang, Tommy Tang, Yifan Ding, Sekhar C Tatikonda, Nicha Dvornek, Xenophon Papademetris, and James Duncan. Adabelief optimizer: Adapting stepsizes by the belief in observed gradients. *Advances in neural information processing systems*, 33:18795–18806, 2020.
- Weidong Zou, Yuanqing Xia, Bineng Zhong, and Weipeng Cao. Adaswitch: Adapting switch from adam to sgdm by exponential function. *Applied Soft Computing*, pp. 114459, 2025.